

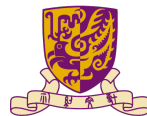
Network Programming

Distributed Storage System: Dynamo

Minchen Yu

SDS@CUHK-SZ

Spring 2026



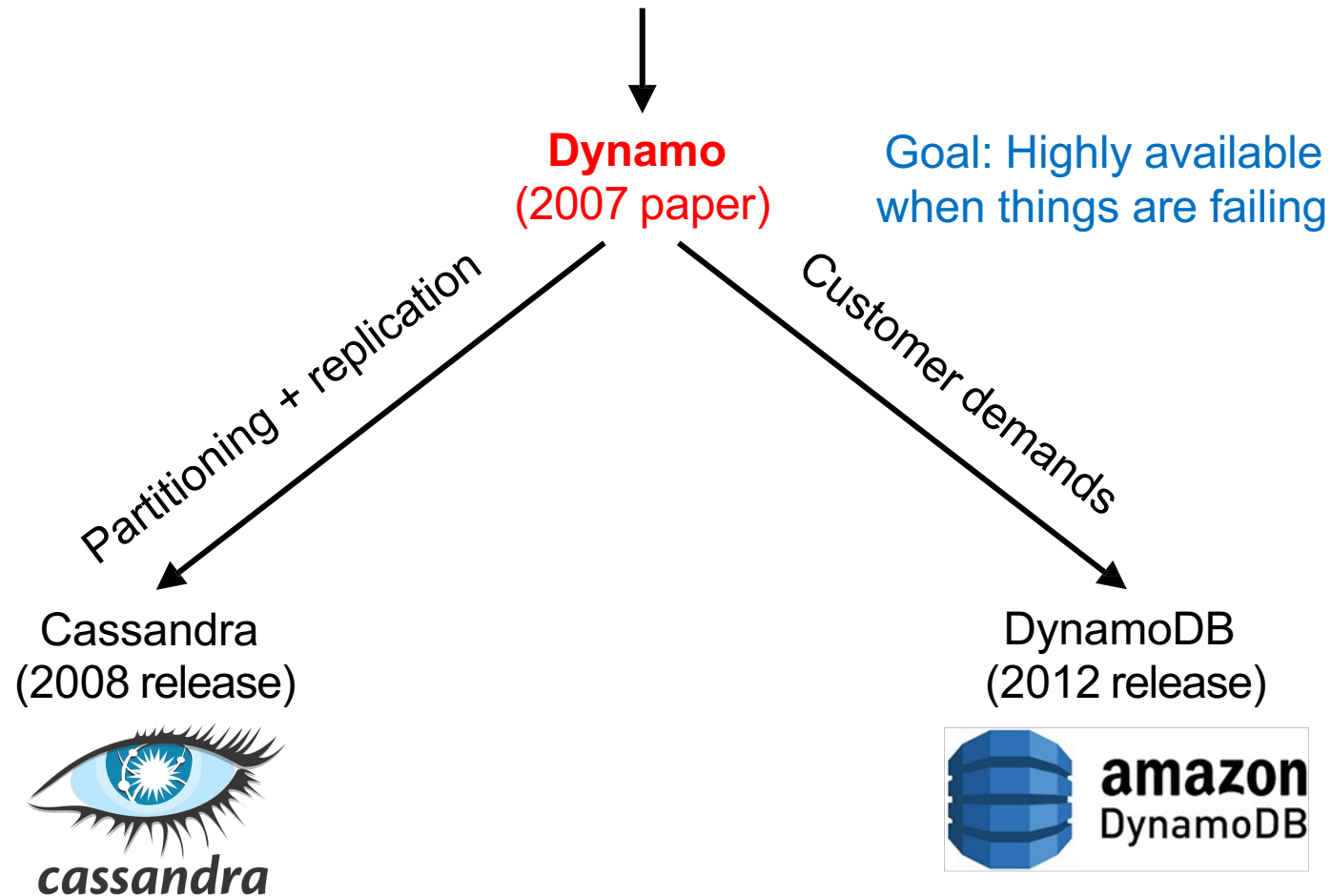
香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen



SCHOOL OF
DATA SCIENCE
數據科學學院

Dynamo impact

Motivation: Workloads like shopping cart do not need the SQL level of complexity and transaction guarantee!



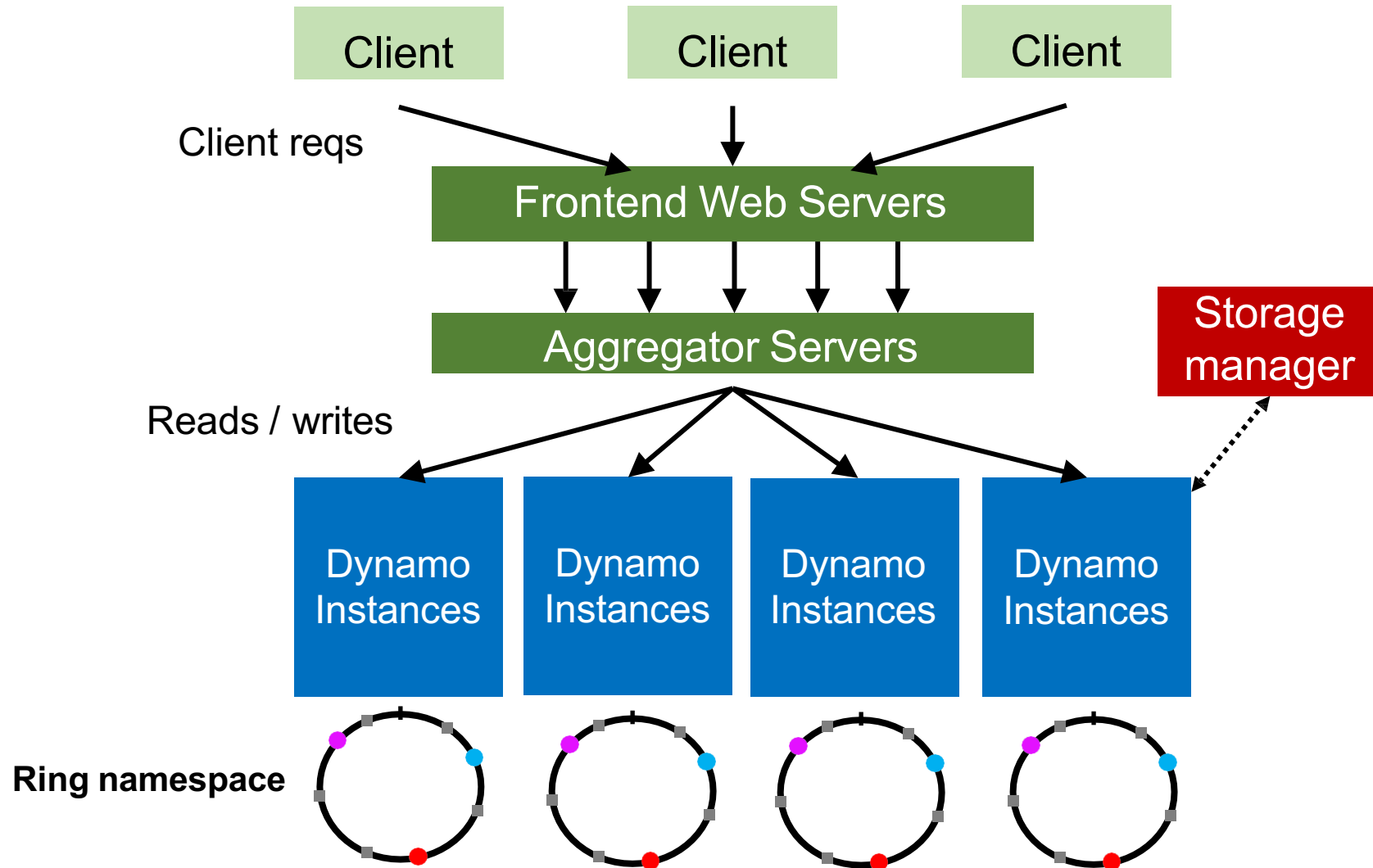
How does Amazon use Dynamo?

- Shopping cart
- Need a data storage to store lots of states
 - Product list (mostly read-only, replicated for high throughput)
 - Recently visited products
 - Orders

Dynamo requirements

- **Highly available writes** despite failures
 - Despite disks failing, network routes flapping, “data centers destroyed by tornadoes”
 - Always respond quickly, even during failures → replication
- **Low request-response latency**: focus on **99.9%** SLA
- **Incrementally scalable** as servers grow to workload
 - Adding “nodes” should be seamless
- Comprehensible **conflict resolution**
 - High availability in above sense implies conflicts

Amazon Dynamo architecture

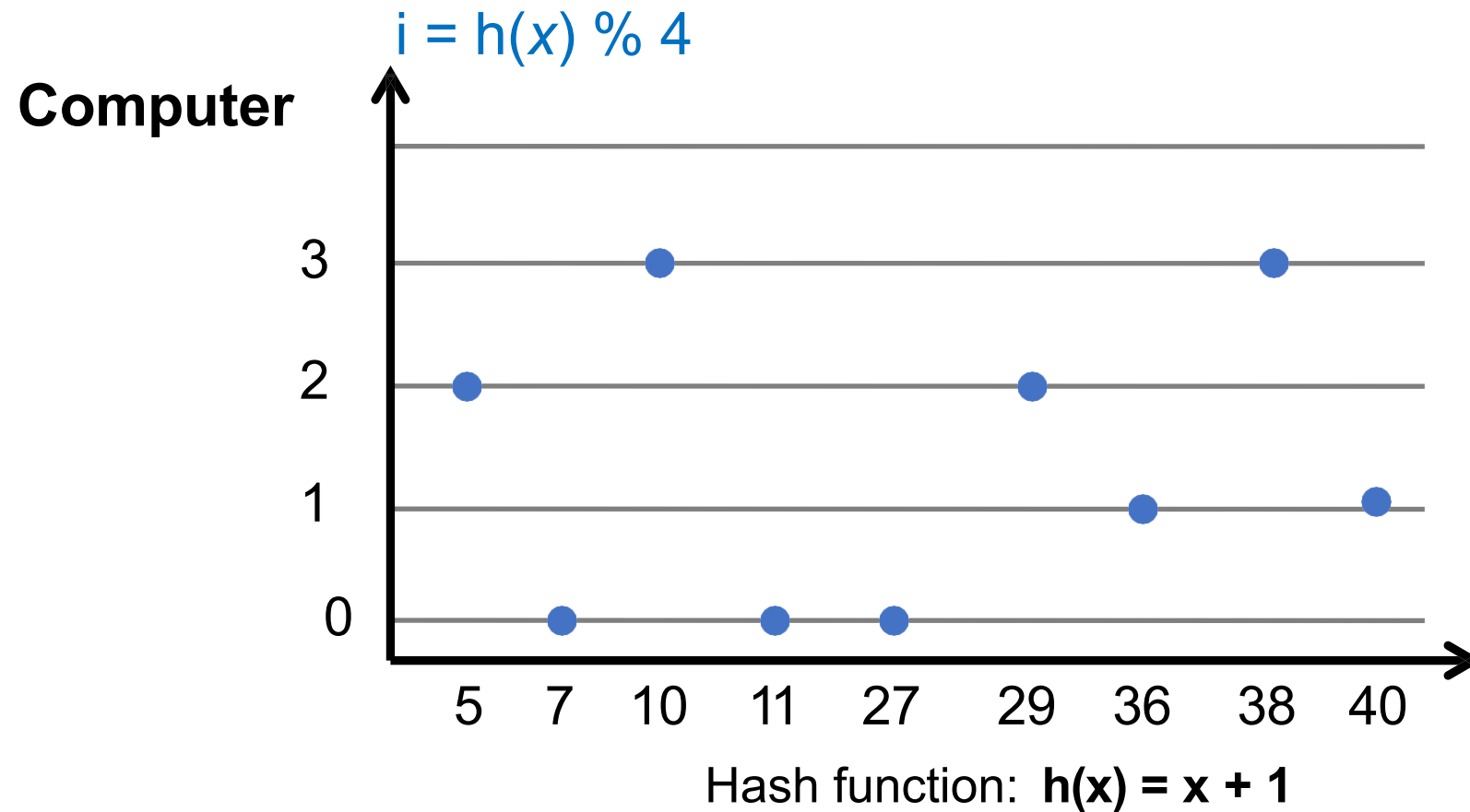


Partitioning approaches

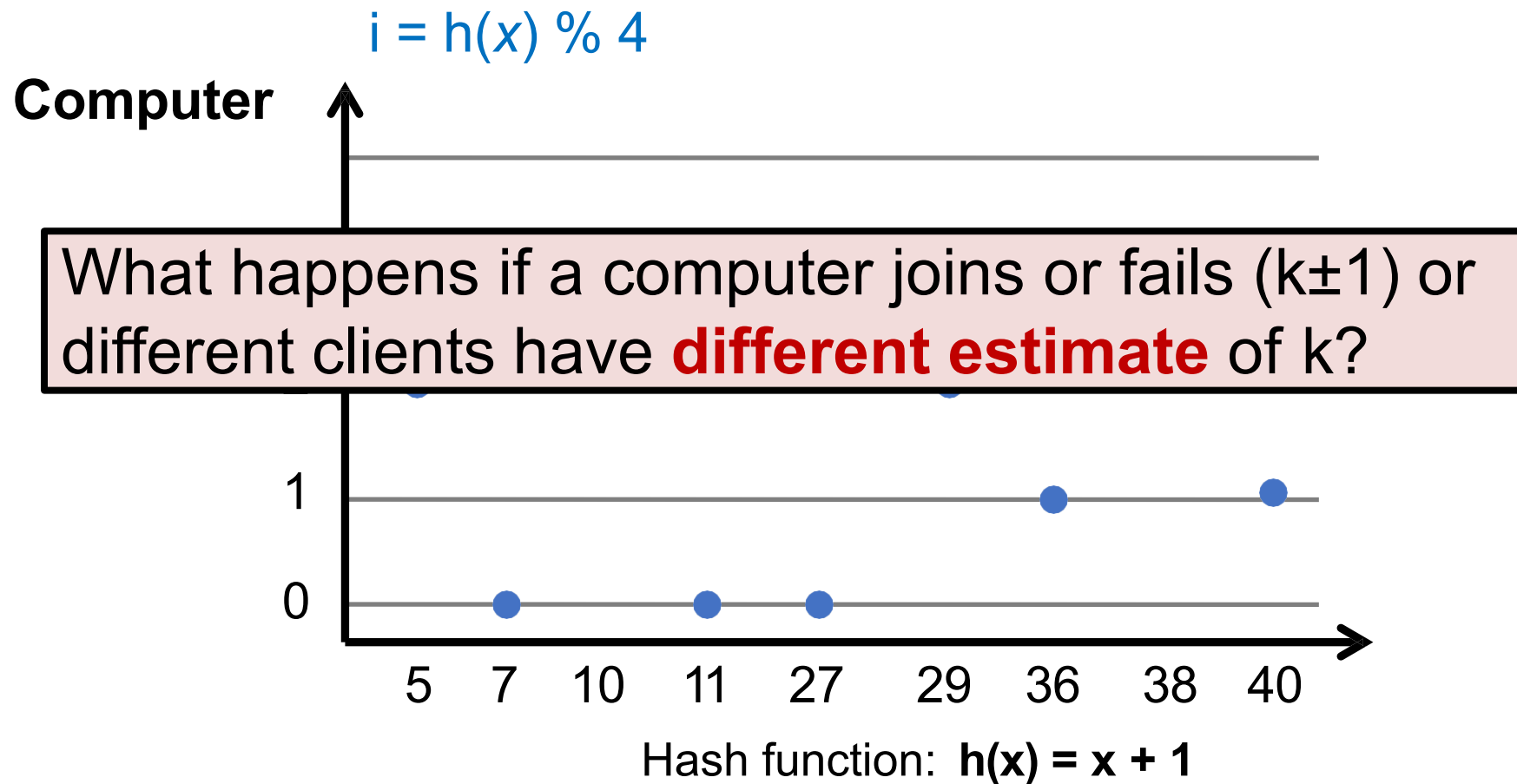
Given many machines and a piece of data, how do we decide where it should live?

- Hash partitioning
 - $\text{Partition} = \text{hash}(\text{key}) \% \text{partition_count}$
- Consistent hashing
 - Dynamo uses this

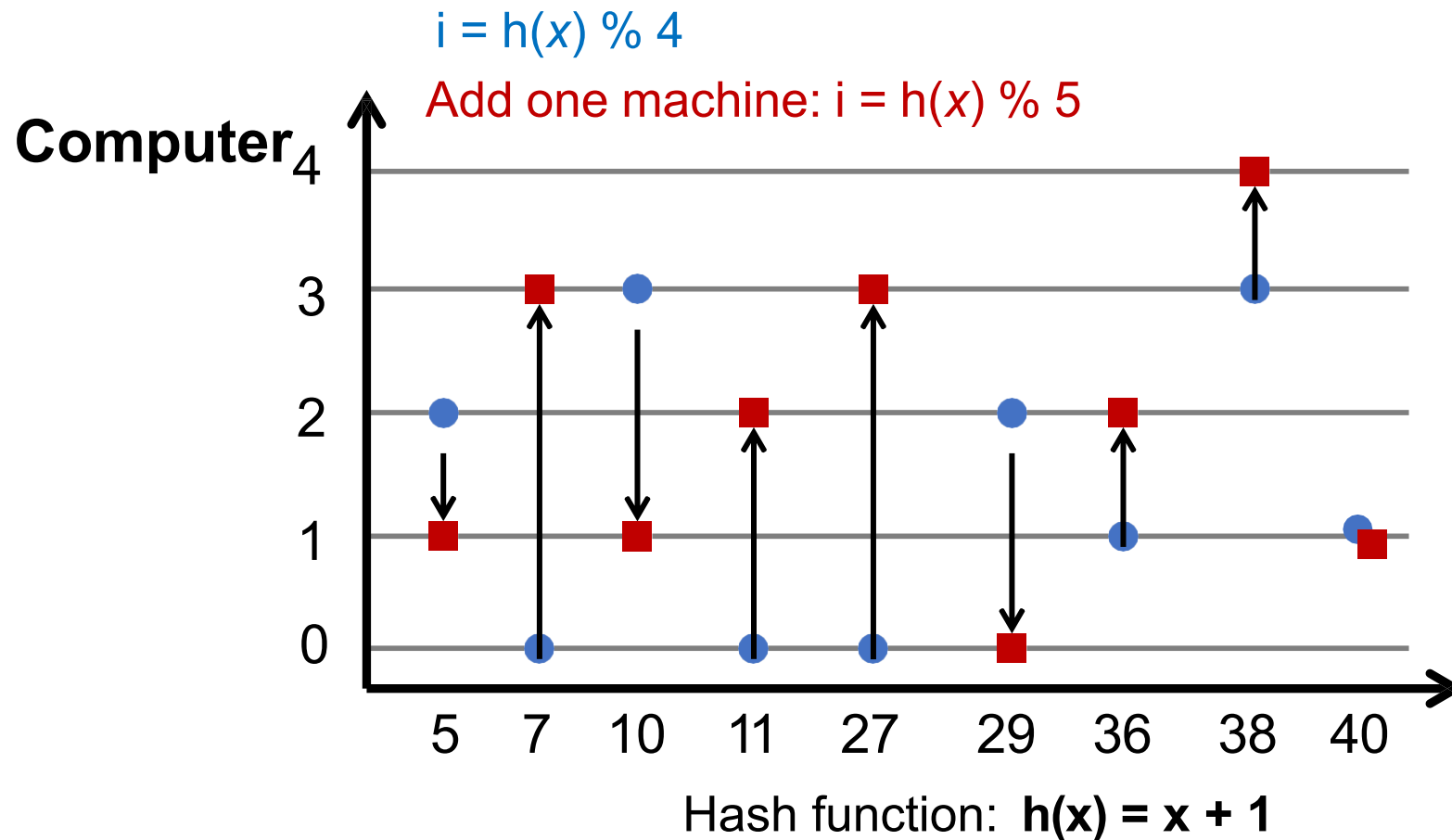
Problem for hash partitioning: Changing number of computers



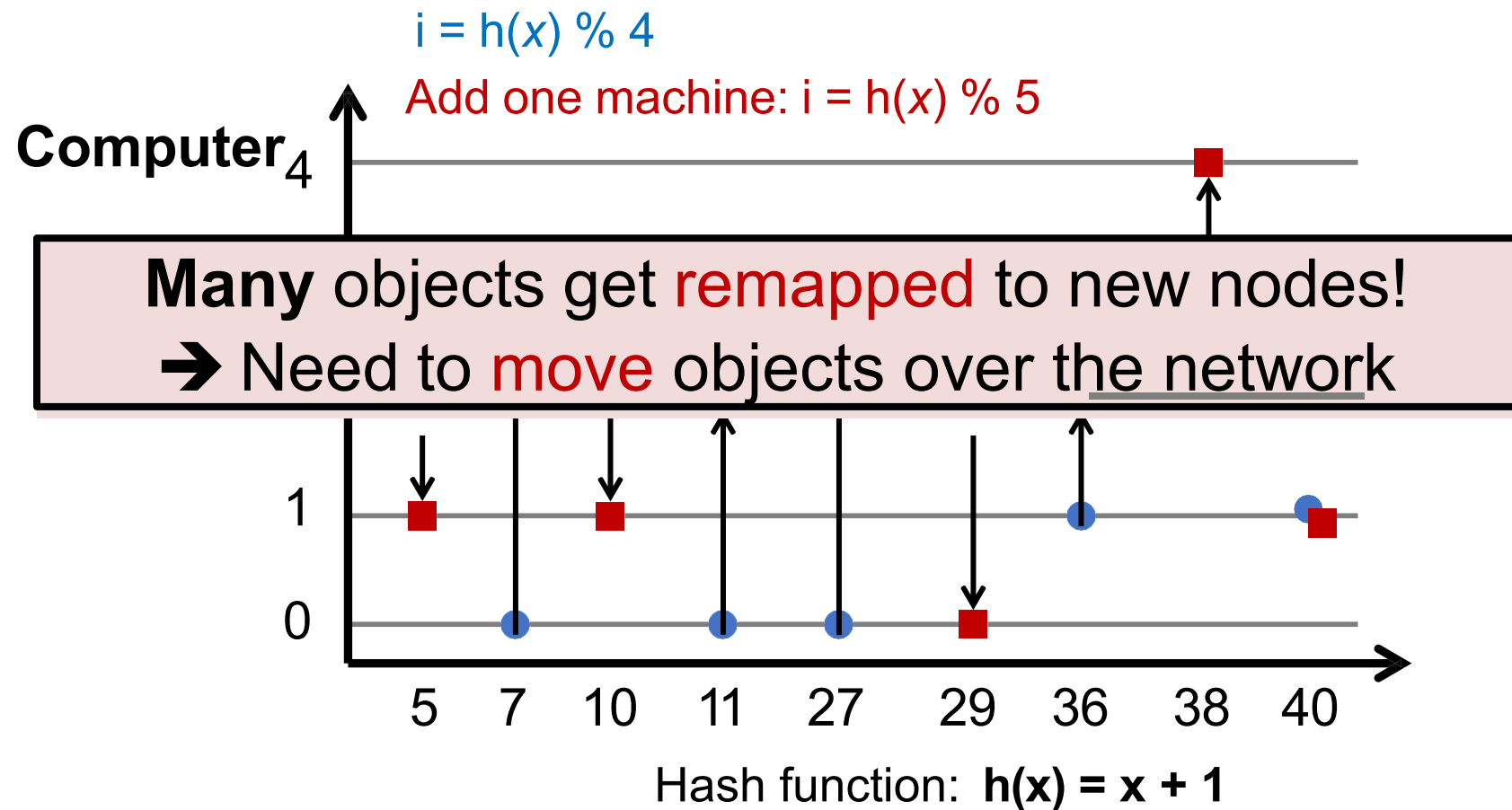
Problem for hash partitioning: Changing number of computers



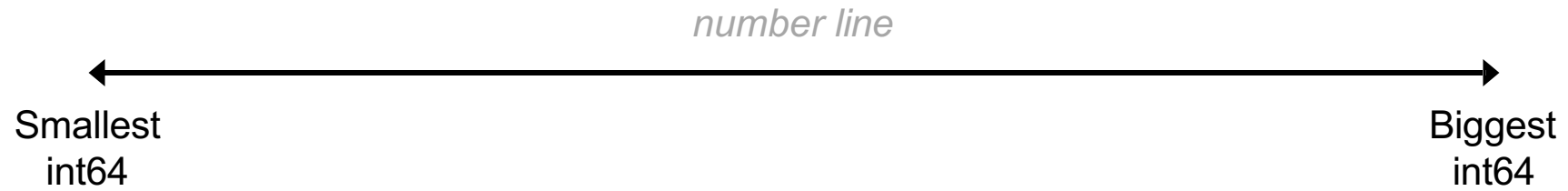
Problem for hash partitioning: Changing number of computers



Problem for hash partitioning: Changing number of computers



Consistent hashing



Consistent hashing

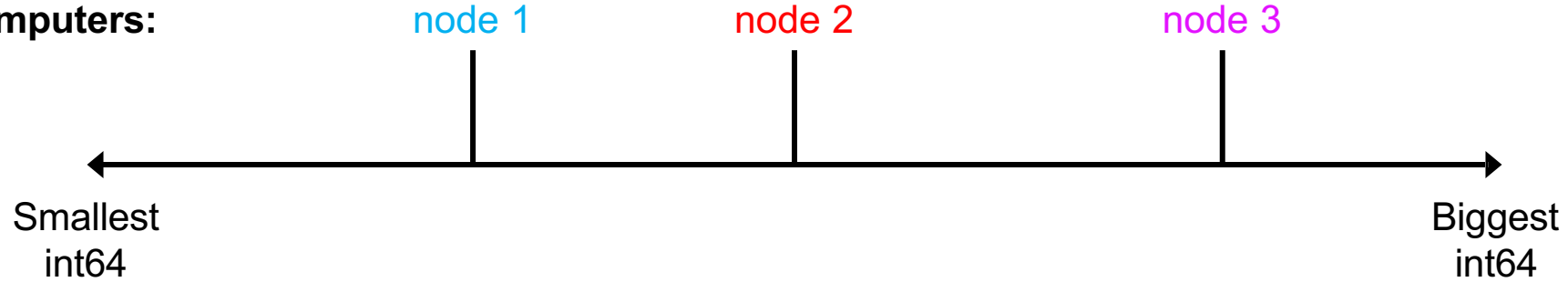
Token map:

token(node1) = pick something

token(node2) = pick something

token(node3) = pick something

Computers:



Assign every **computer** a point on the number line.
Could be random (though newer approaches are cleverer).

No hashing needed, yet!

Consistent hashing

Token map:

token(node1) = pick something

token(node2) = pick something

token(node3) = pick something

Computers:

node 1

node 2

node 3

Smallest
int64

Biggest
int64

Rows:

A

B

C

D

E

Assign each **row** a point on the number line.

$\text{token}(\text{row}) = \text{hash}(\text{row's partition key})$

Consistent hashing

Token map:

token(node1) = pick something

token(node2) = pick something

token(node3) = pick something

Computers:

node 1

node 2

node 3

Rows:

A

B

C

D

E

node 1

node 2

node 3

Cluster:

A

B

C

D

Each node's token is the **inclusive end** of a range. A row is mapped to a node based on the range it is in.

Consistent hashing

Token map:

token(node1) = pick something

token(node2) = pick something

token(node3) = pick something

Computers:

node 1

node 2

node 3

Rows:

A

B

C

D

E

tokens > biggest node token are in the **wrapping range**. Rows in this region go to the node with the smallest token (i.e., a ring space).

node 1

node 2

node 3

A

E

B

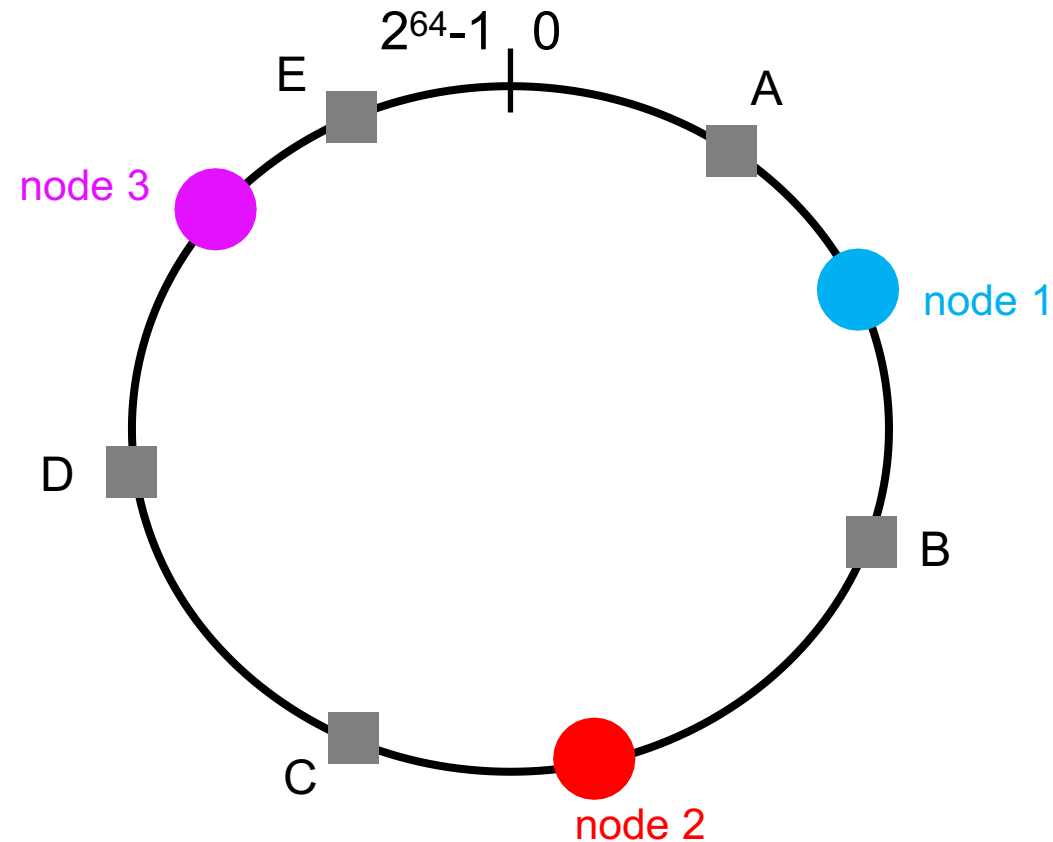
C

D

Cluster:

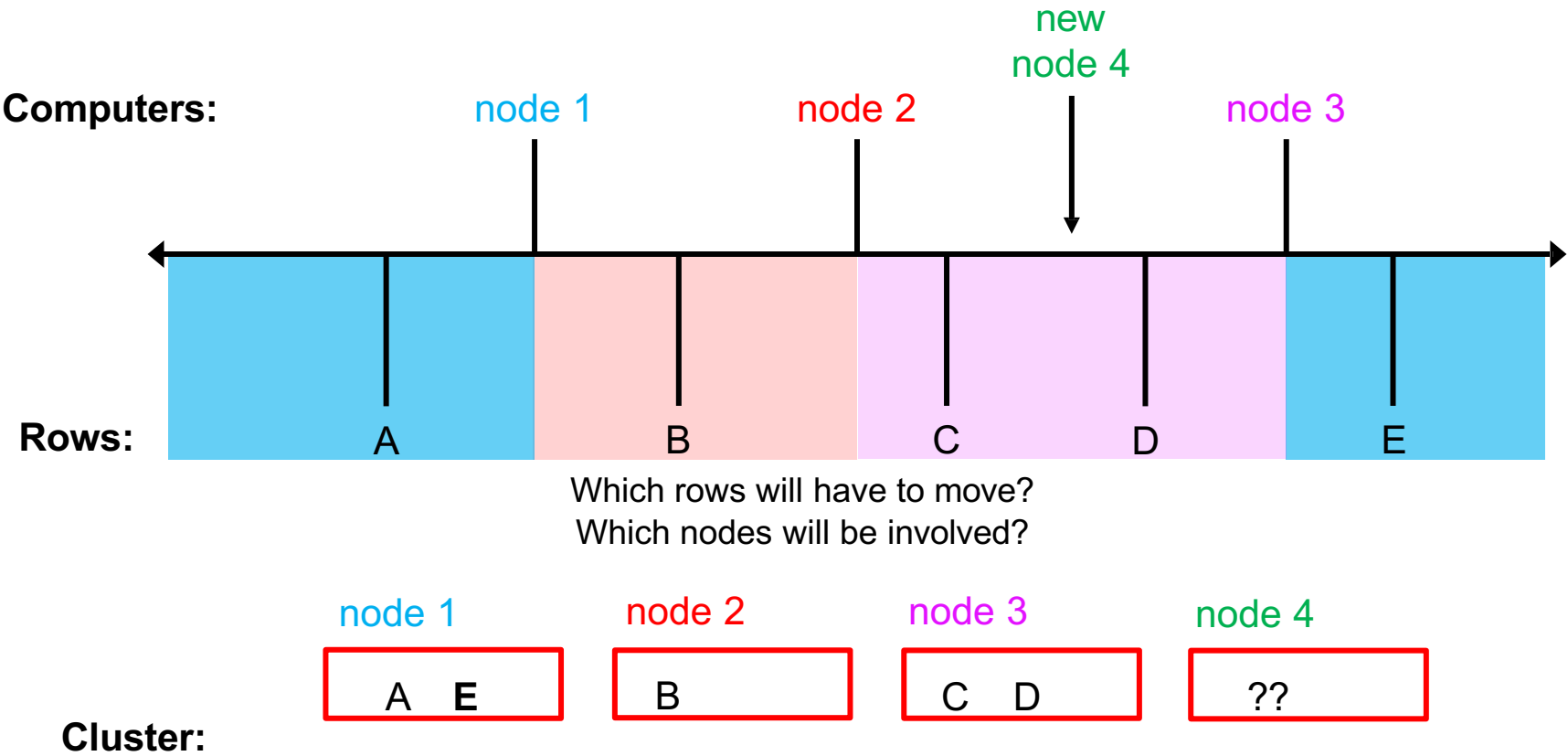
Alternate ring-based visualization

Given the wrapping, clusters using consistent hashing form a “token ring”



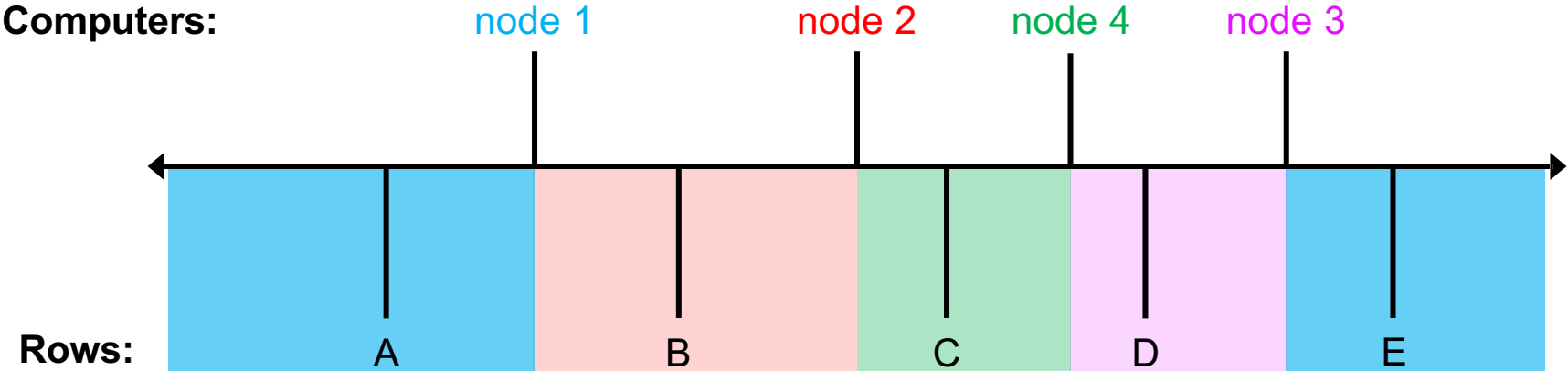
Adding a node

Token map:
token(node1) = pick something
token(node2) = pick something
token(node3) = pick something
token(node4) = pick something



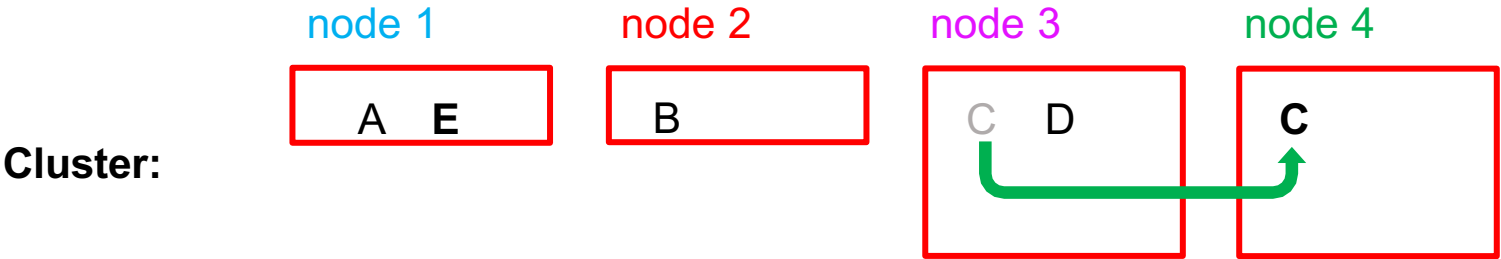
Adding a node

Token map:
token(node1) = pick something
token(node2) = pick something
token(node3) = pick something
token(node4) = pick something



Which rows will have to move? **Only C**

Which nodes will be involved? Only node 3 and node 4



Adding a node

Token map:

token(node1) = pick something

token(node2) = pick something

token(node3) = pick something

token(node4) = pick something

Computers:

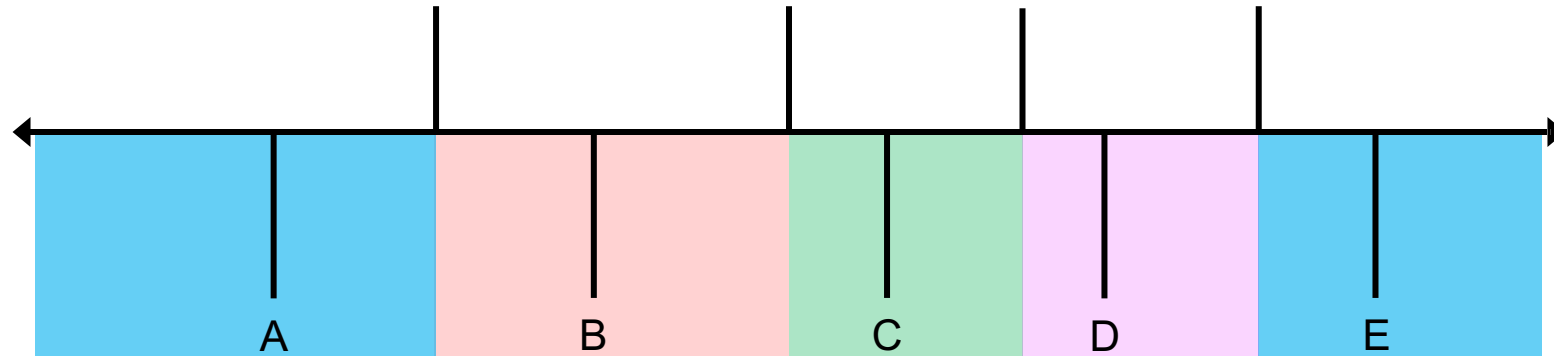
node 1

node 2

node 4

node 3

Rows:



Typically, what fraction of the data must move when we scale from $N-1$ to N ?

Hash partitioning: about $(N-1) / N$ of the data

Consistent hashing: about $(\text{size of new range}) / (\text{size of all ranges})$ of the data must move.

Problems

Token map:

token(node1) = pick something

token(node2) = pick something

token(node3) = pick something

token(node4) = pick something

Computers:

node 1

node 2

node 4

node 3

Rows:

A

B

C

D

E

Problems with adding node 4:

- **Long term:** Only load of node 3 is alleviated
- **Short term:** Node 3 bears all the burden of transferring data to node 4

Solution: introducing vnodes (virtual nodes)

Virtual nodes (vnodes)

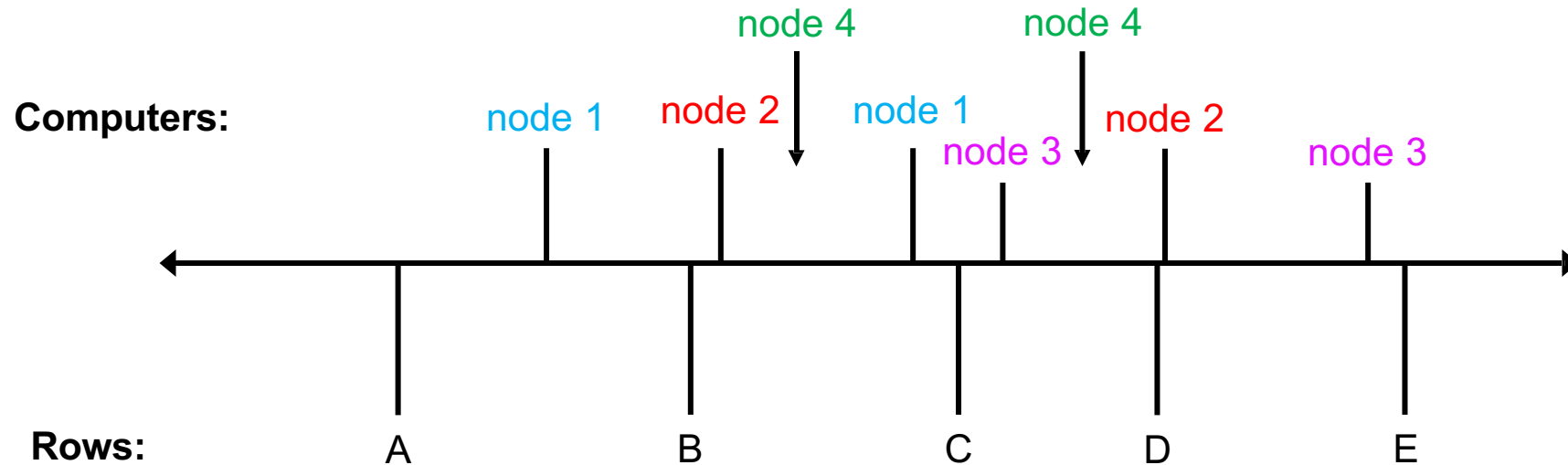
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



Each (physical) node is responsible for multiple ranges (in this case, 2)

- How many vnodes per node is configurable
- Node 4 will share some load off node 1 and node 2
- Achieves better (short-term and long-term) load balancing with a larger number of vnodes

Heterogeneity

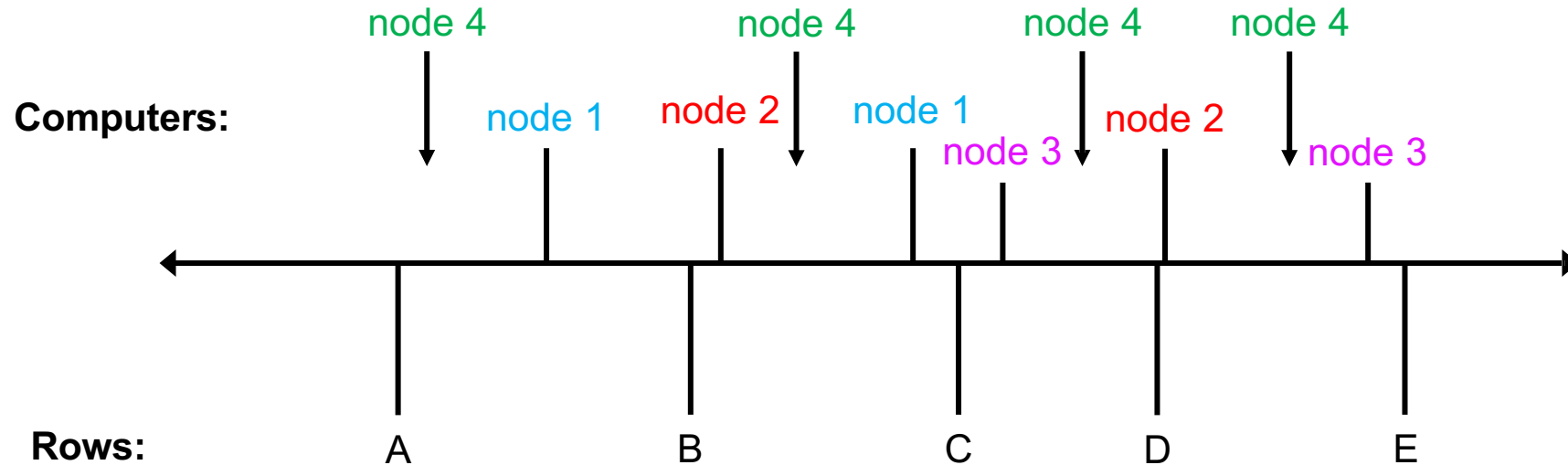
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8, t9, t10}



Heterogeneity: Some machines (e.g., newer ones) have more resources

- More powerful nodes can have more capacity, thus more vnodes
- Probabilistically, they'll do more work and store more data

Token map storage

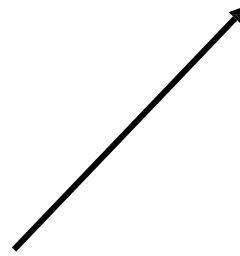
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}

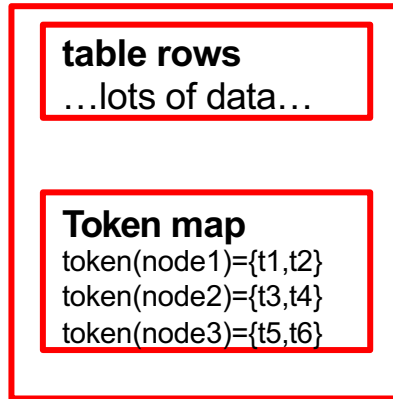


Where should this live?

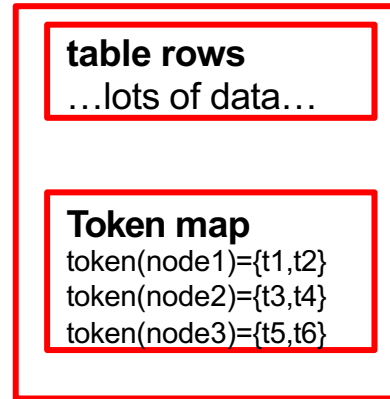
We don't want a single point of failure

Token map storage

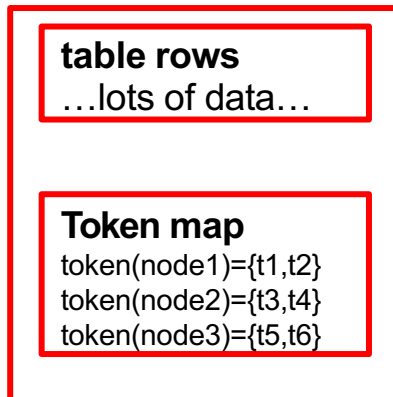
node 1



node 2



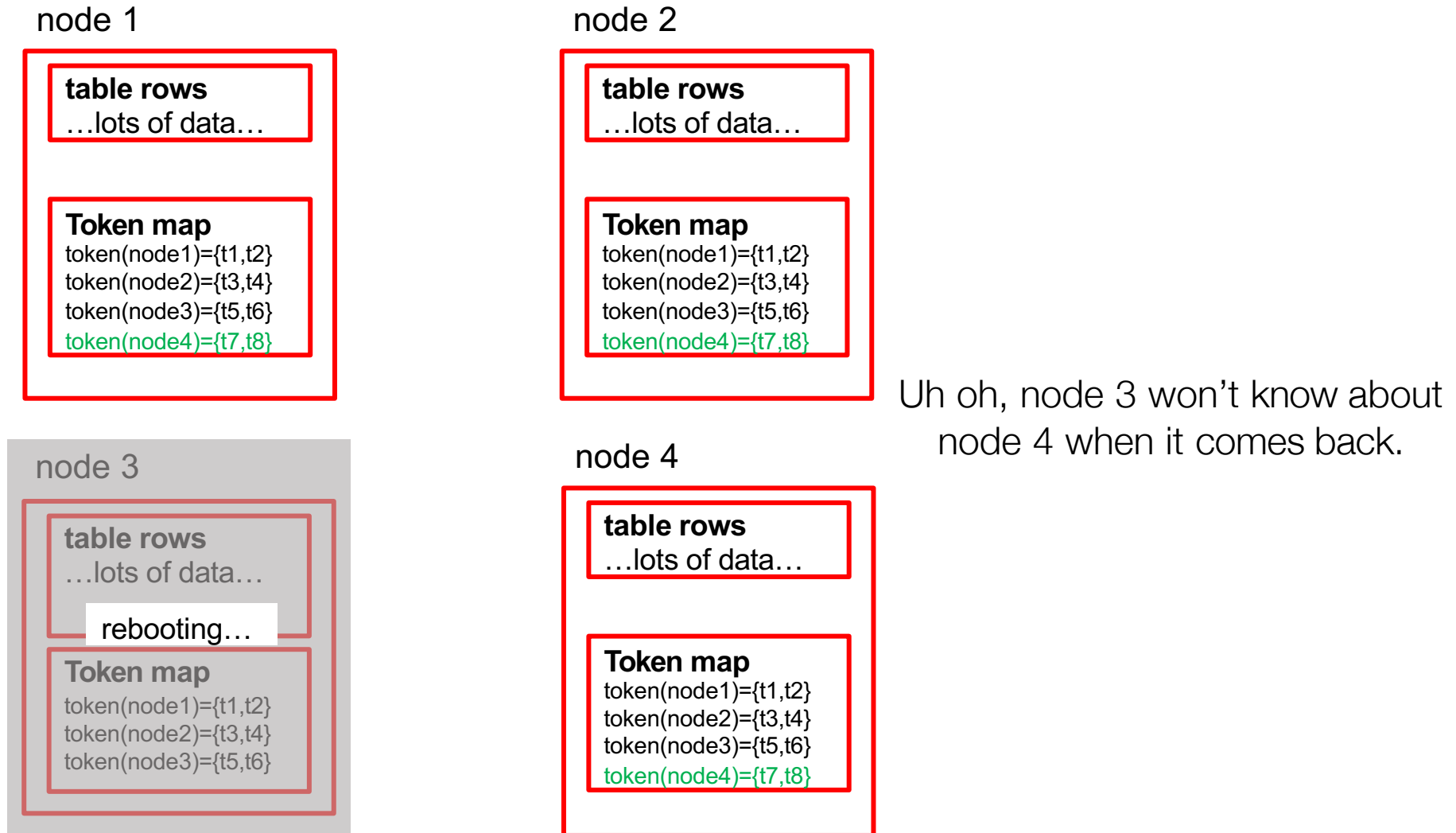
node 3



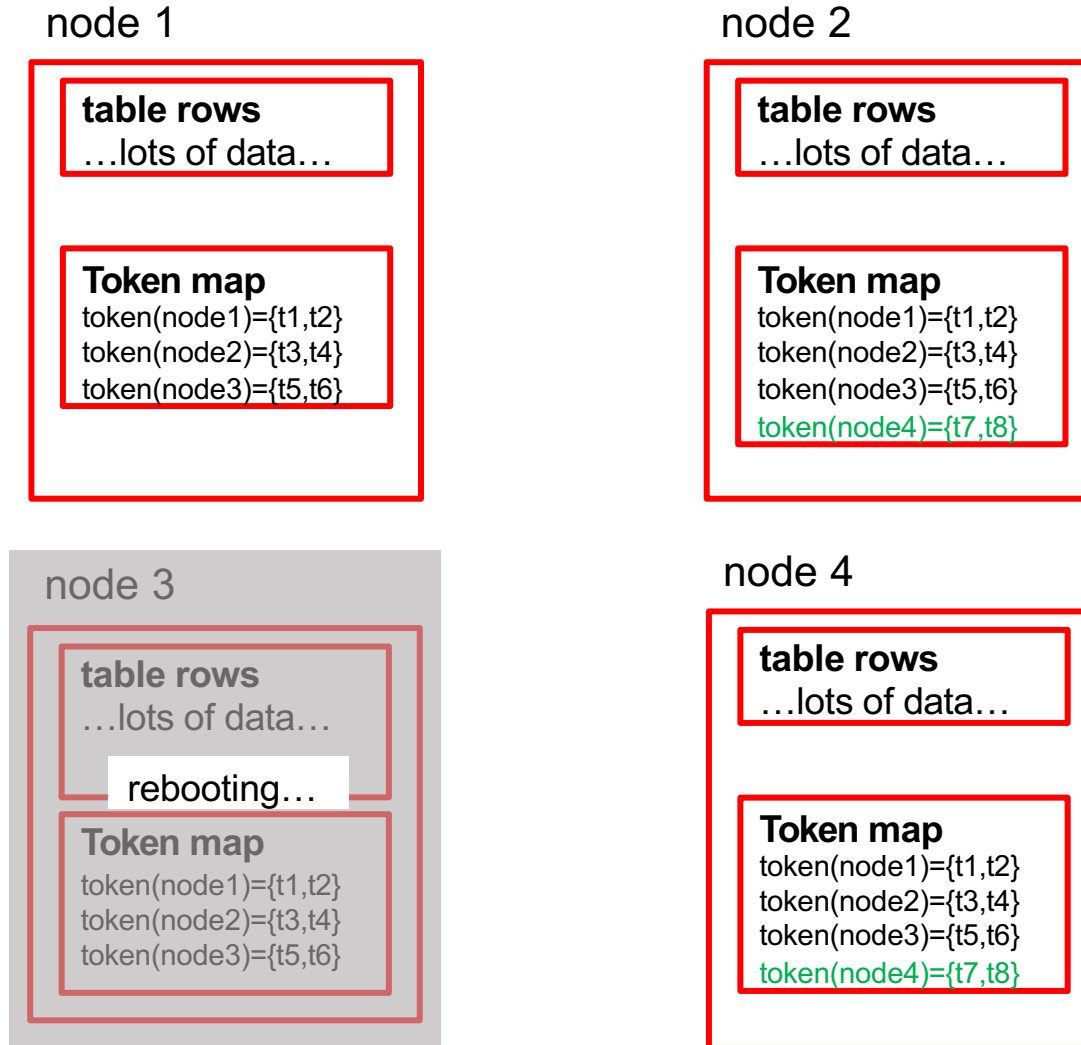
Every node has a copy of the global token map.

They should all get updated when new nodes join.

Adding nodes: Bad approach

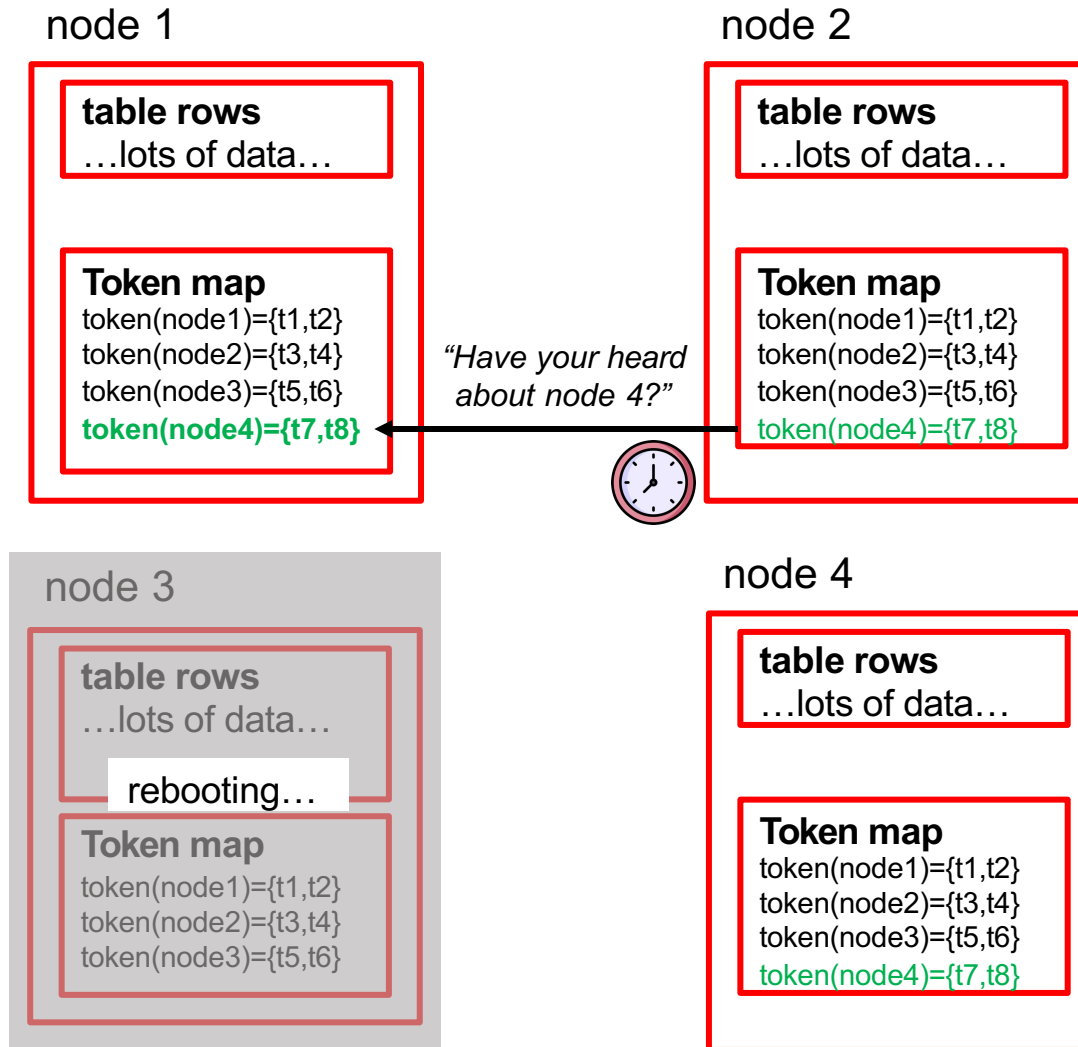


Better approach: Gossip



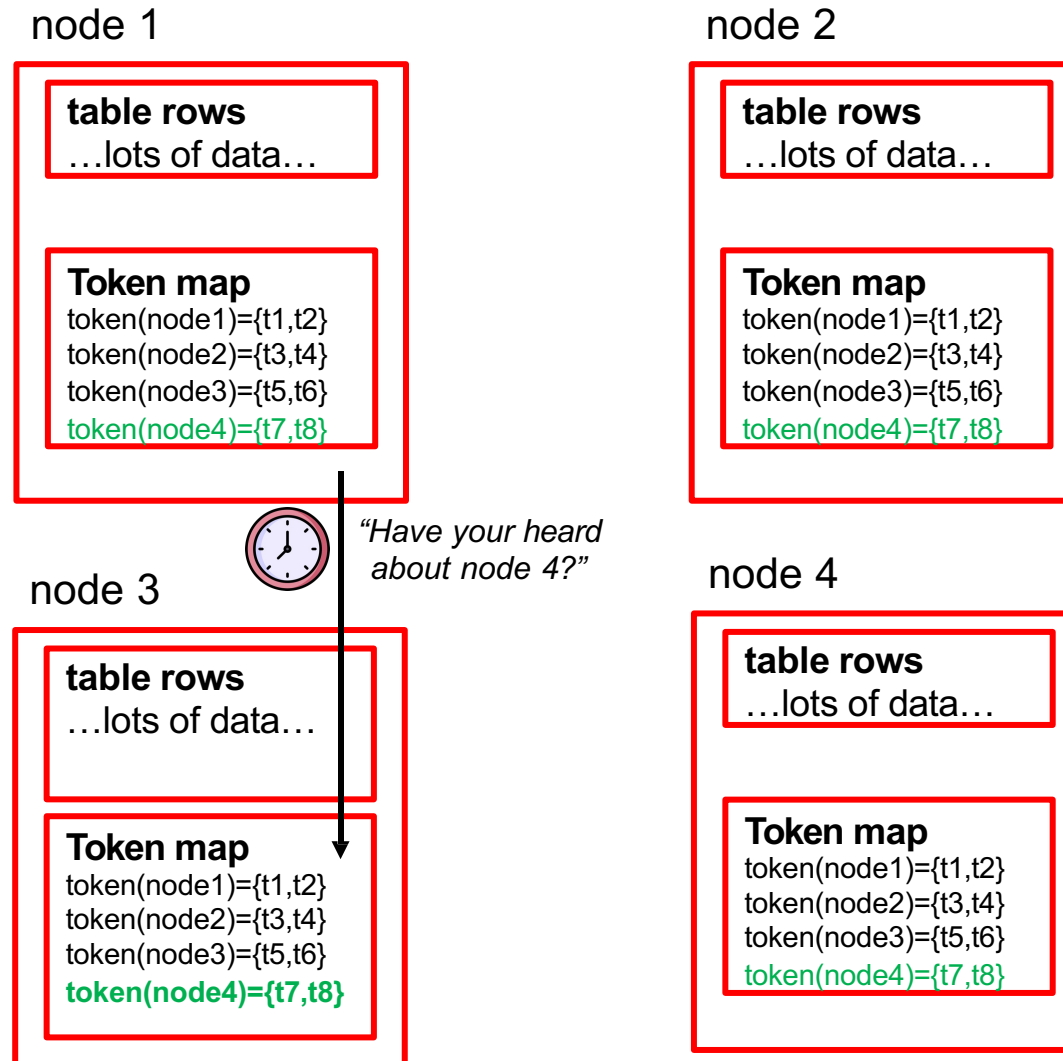
Just inform one or a few nodes about the new node

Better approach: Gossip



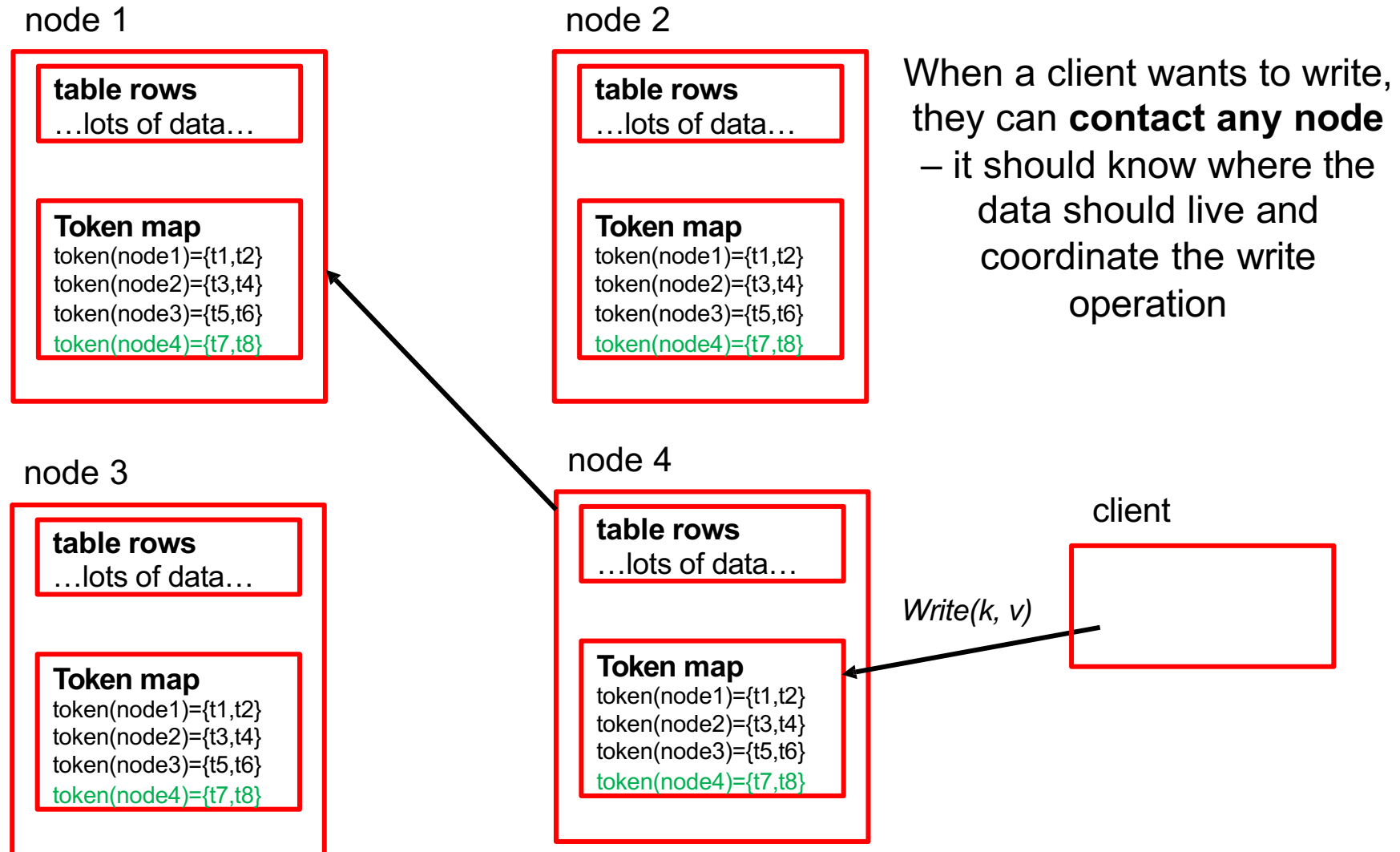
Once per second:
Choose a random friend,
Gossip about new nodes

Better approach: Gossip



Eventually, every node should find out...

Client can contact any node



Replication

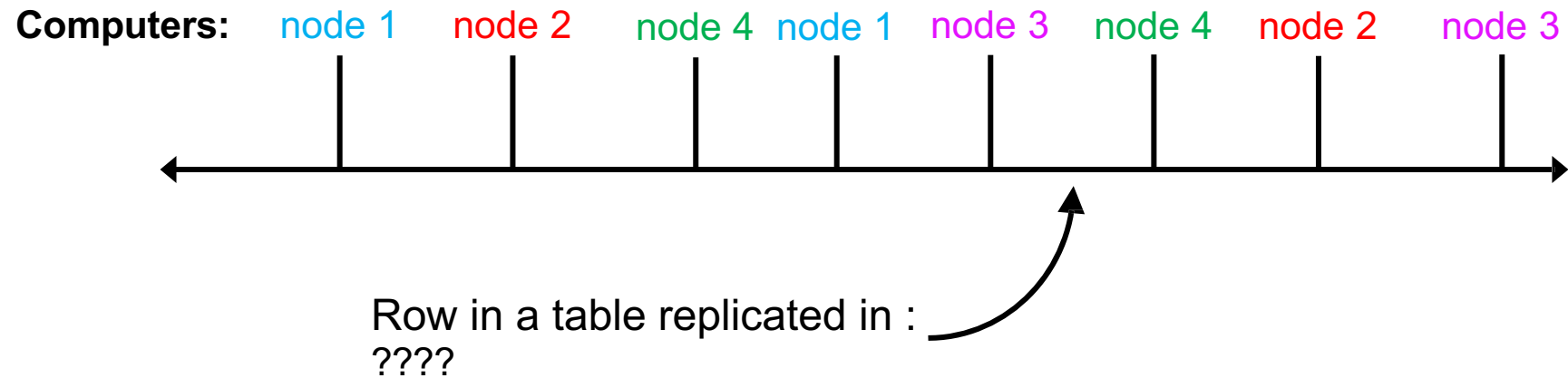
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



Replication factor (RF) of N (where N == 2)

Replication

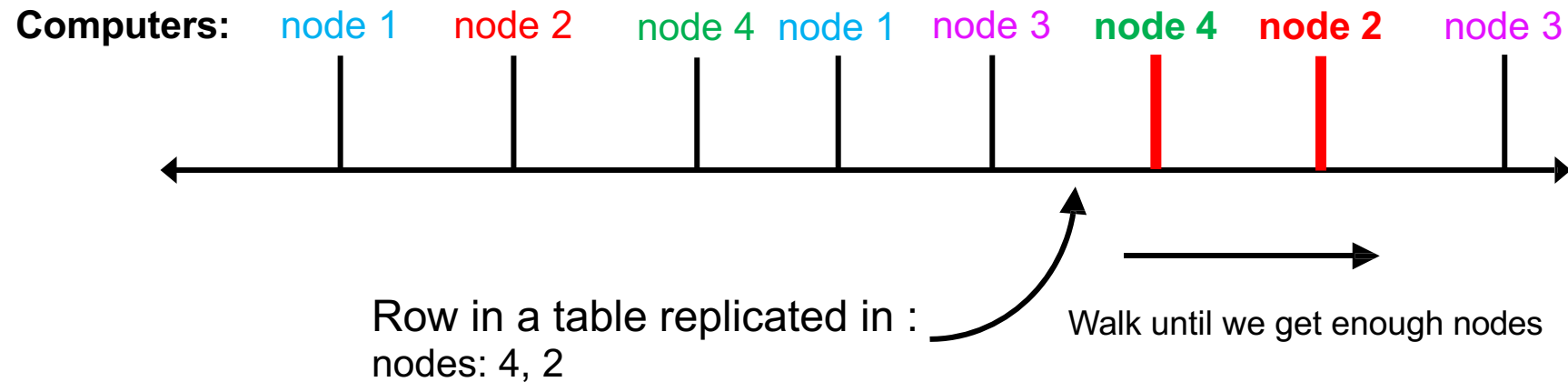
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



$$RF = N \text{ (where } N == 2\text{)}$$

Replication

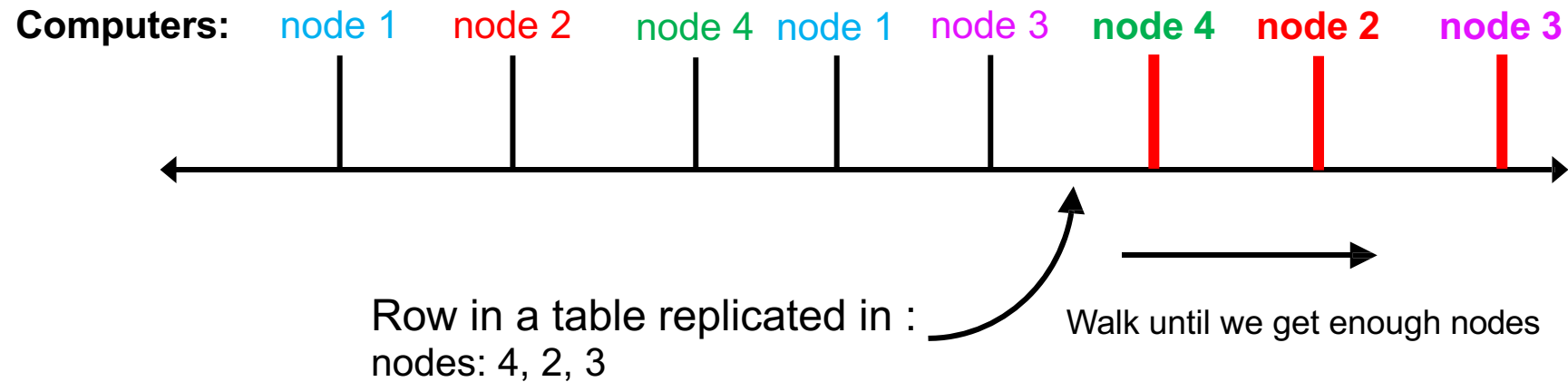
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



RF = N (where N == 3)

Replication

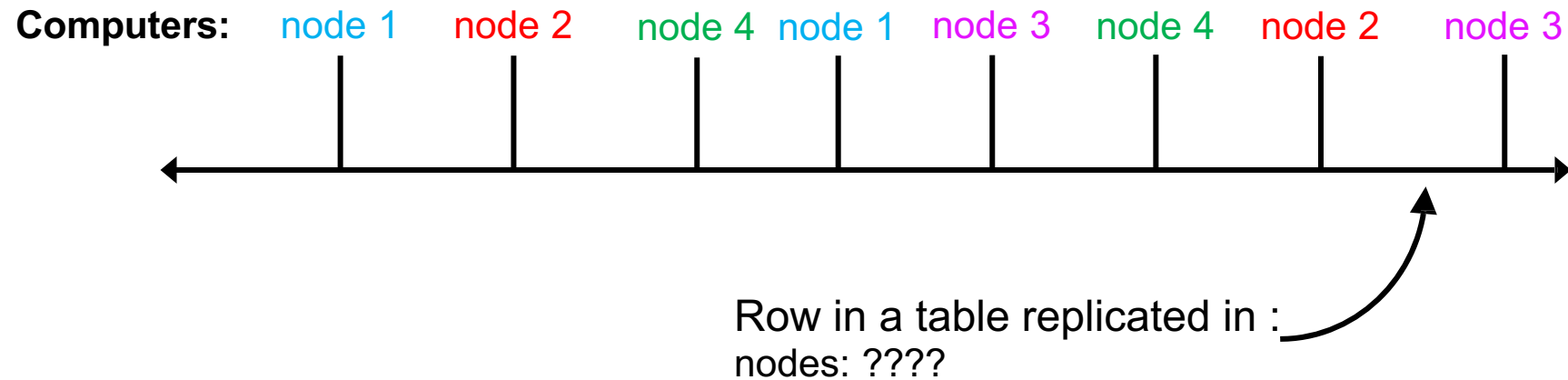
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



RF = N (where N == 3)

Replication

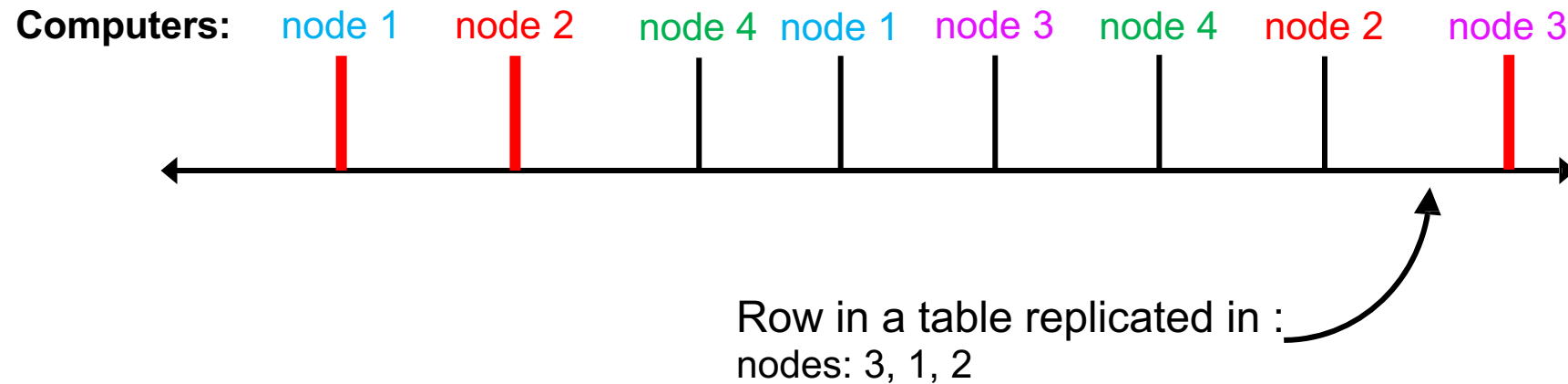
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



Replication factor of N (where N == 3)

Replication

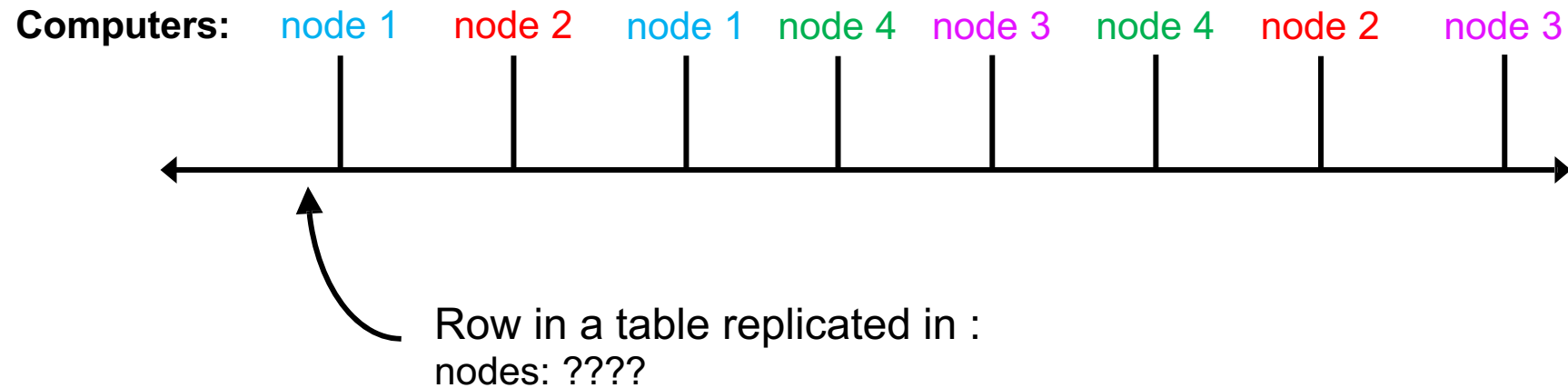
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



RF = N (where N == 3)

Replication

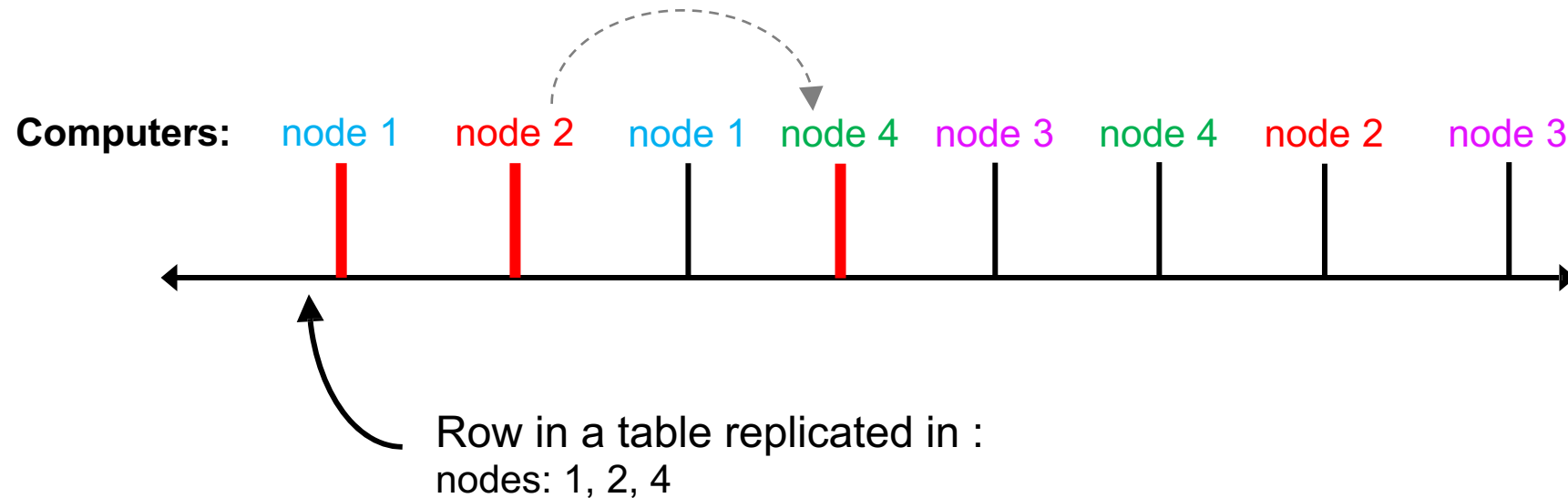
Token map:

token(node1) = {t1, t2}

token(node2) = {t3, t4}

token(node3) = {t5, t6}

token(node4) = {t7, t8}



RF = N (where N == 3)

Important: Keeping multiple copies on vnodes on the same node provides little safety (when a node dies, all its vnodes die). Same **“failure domain”**.

Dynamo **skips** nodes to ensure replicas reside on different nodes.

Write acks

- In distributed storage/database systems, an **ack** means our data is *committed*
- “Committed” means our data is “safe”, even if bad things happen. The definition varies system to system, based on what bad things are considered. For example:
 - A node could hang until rebooted; a node’s disk could permanently fail
 - A rack could lose power; a datacenter could be destroyed

Write acks: WhatsApp example

How to check read receipts

 Android  iOS  KaiOS

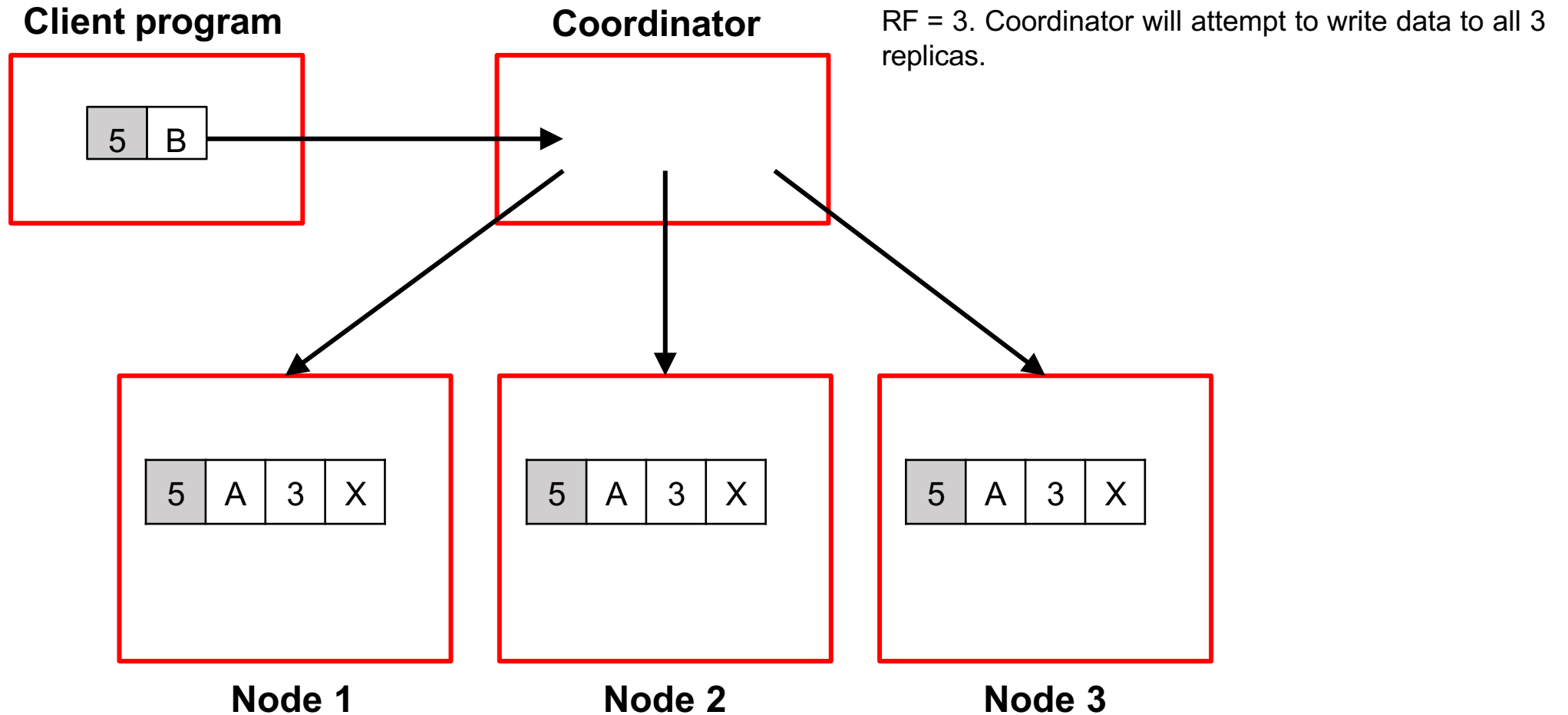
Check marks will appear next to each message you send. Here's what each one means:

- ✓ The message was successfully sent.
- ✓✓ The message was successfully delivered to the recipient's phone or any of their linked devices.
- ✓✓ The recipient has read your message.

These are examples of “**acks**” (acknowledgments)

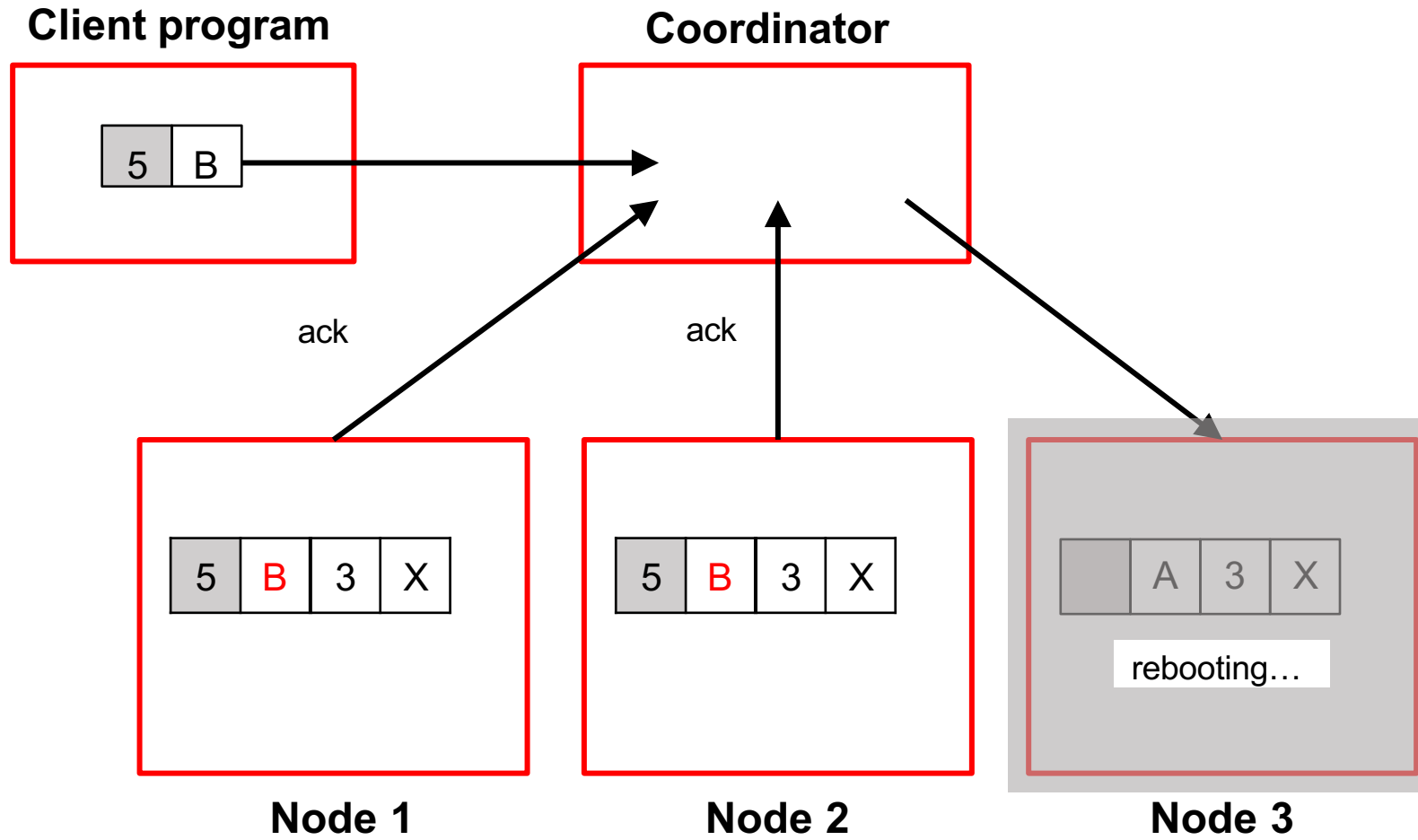
https://faq.whatsapp.com/665923838265756/?cms_platform=android&helpref=platform_switcher

Dynamo writes



Dynamo writes

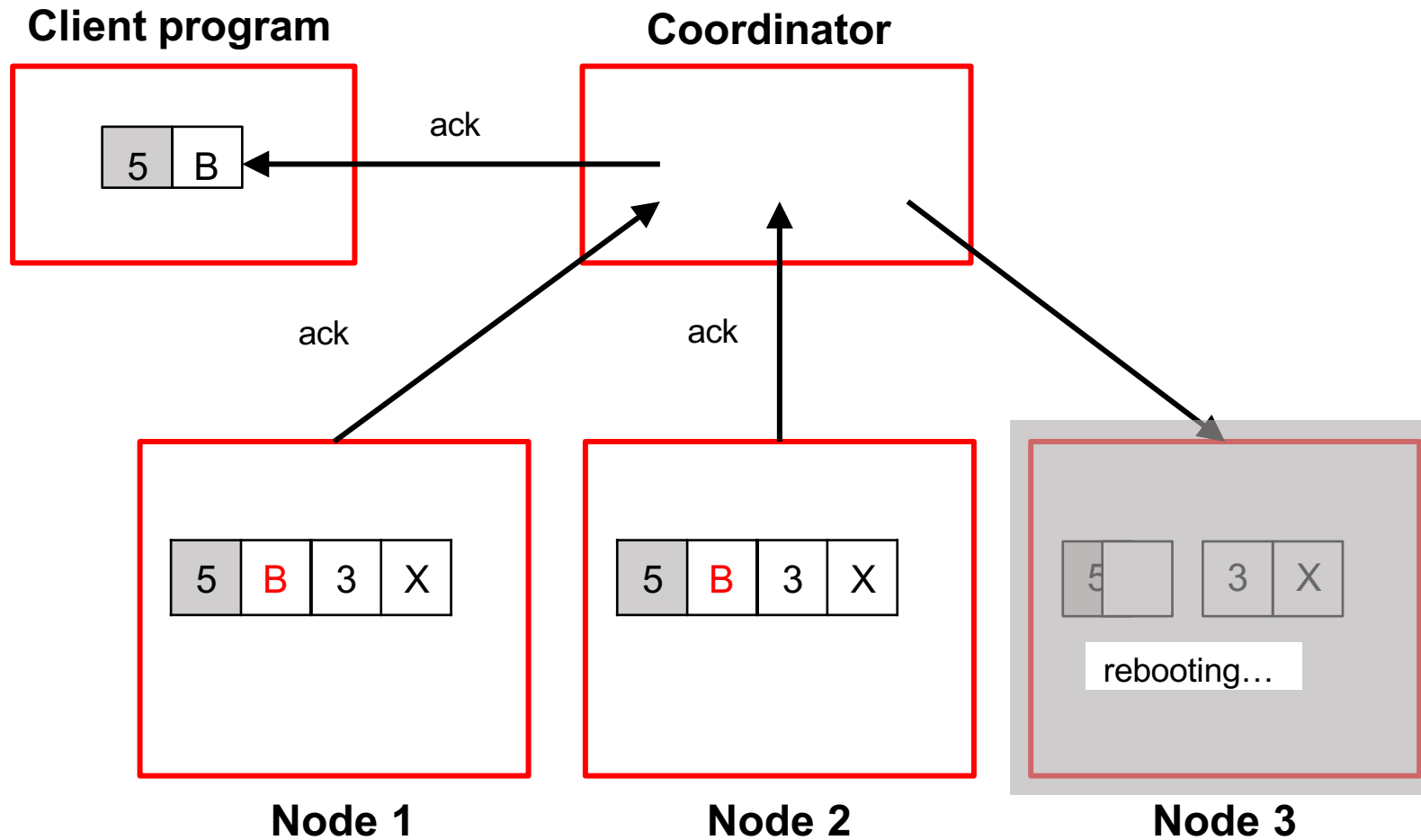
RF = 3. Coordinator will attempt to write data to all 3 replicas.



At what point should we send an ack back to the client?

Dynamo writes

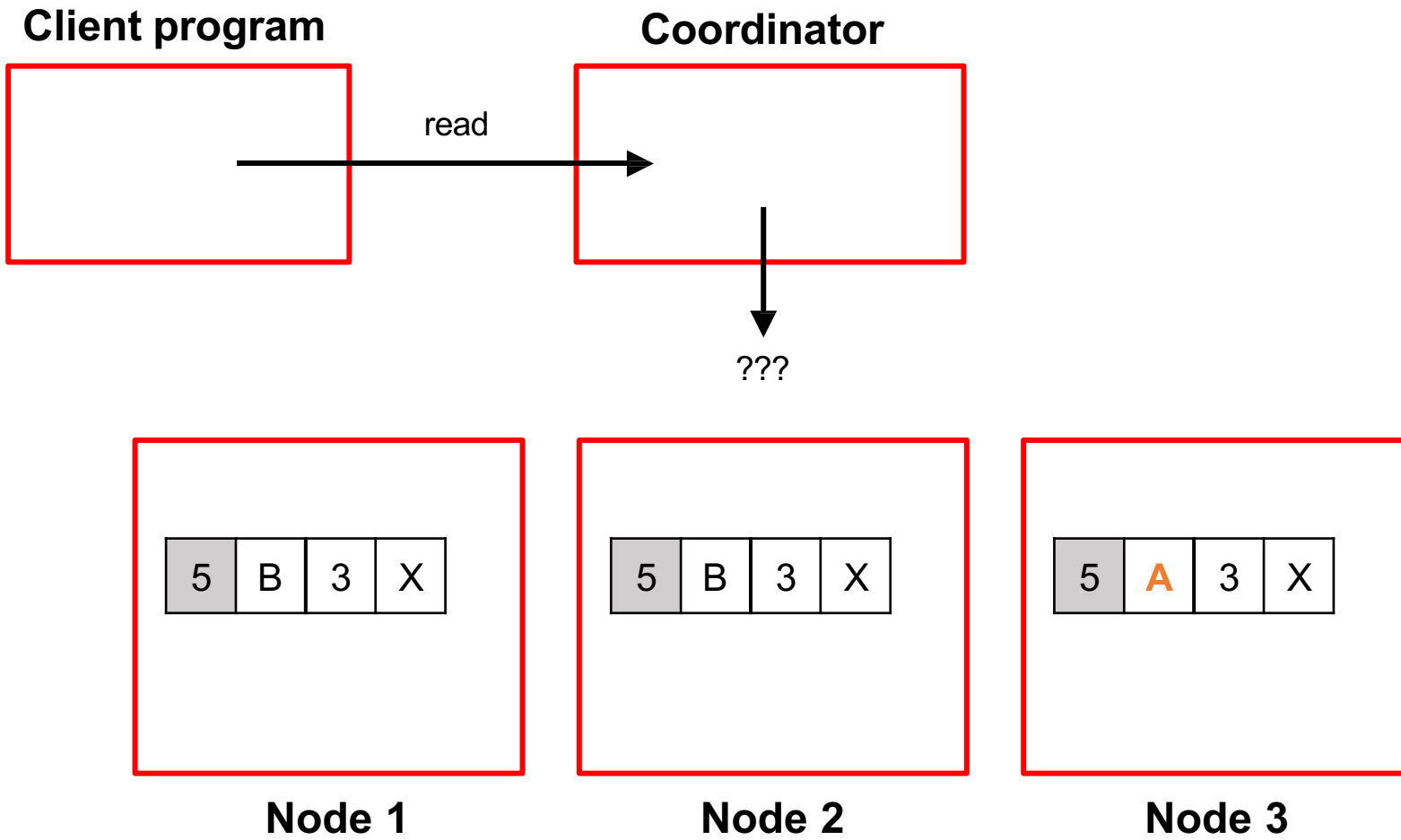
RF = 3. Coordinator will attempt to write data to all 3 replicas.



At what point should we send an ack back to the client?
Configurable: **W = 2** lets coordinator ack now, and data is fairly safe.

Dynamo reads

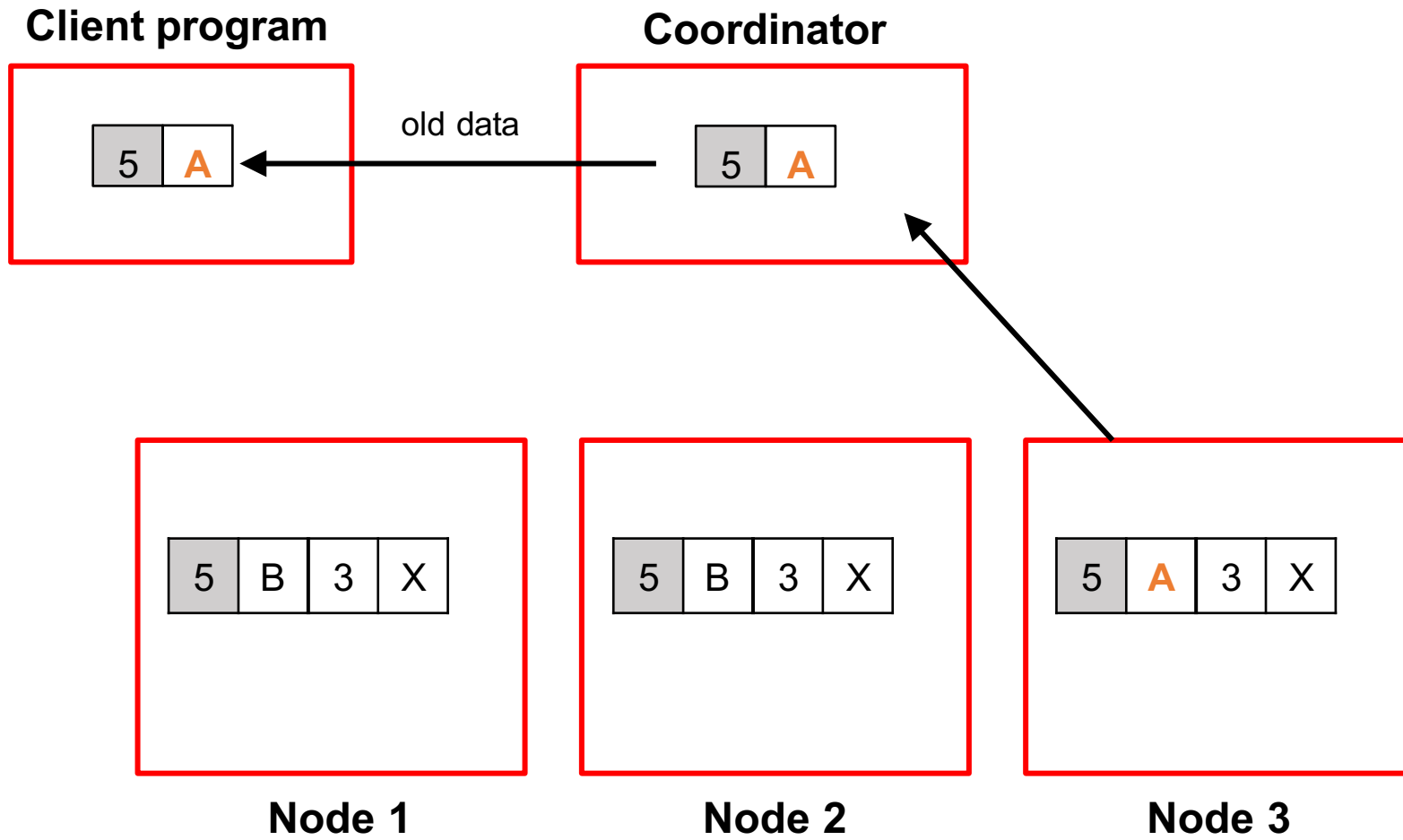
RF = 3



How Dynamo handles read?

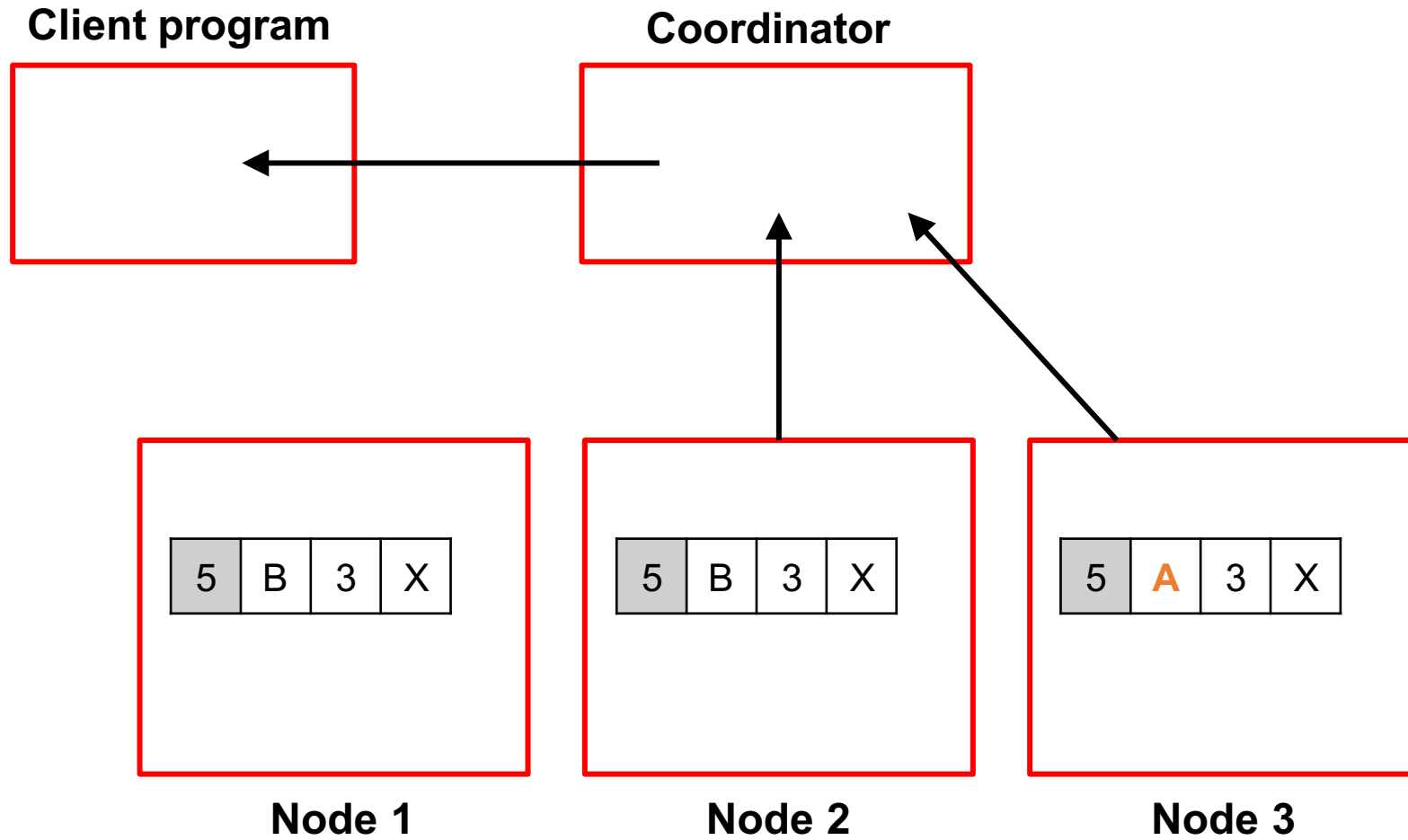
Dynamo reads

RF = 3



Dynamo reads

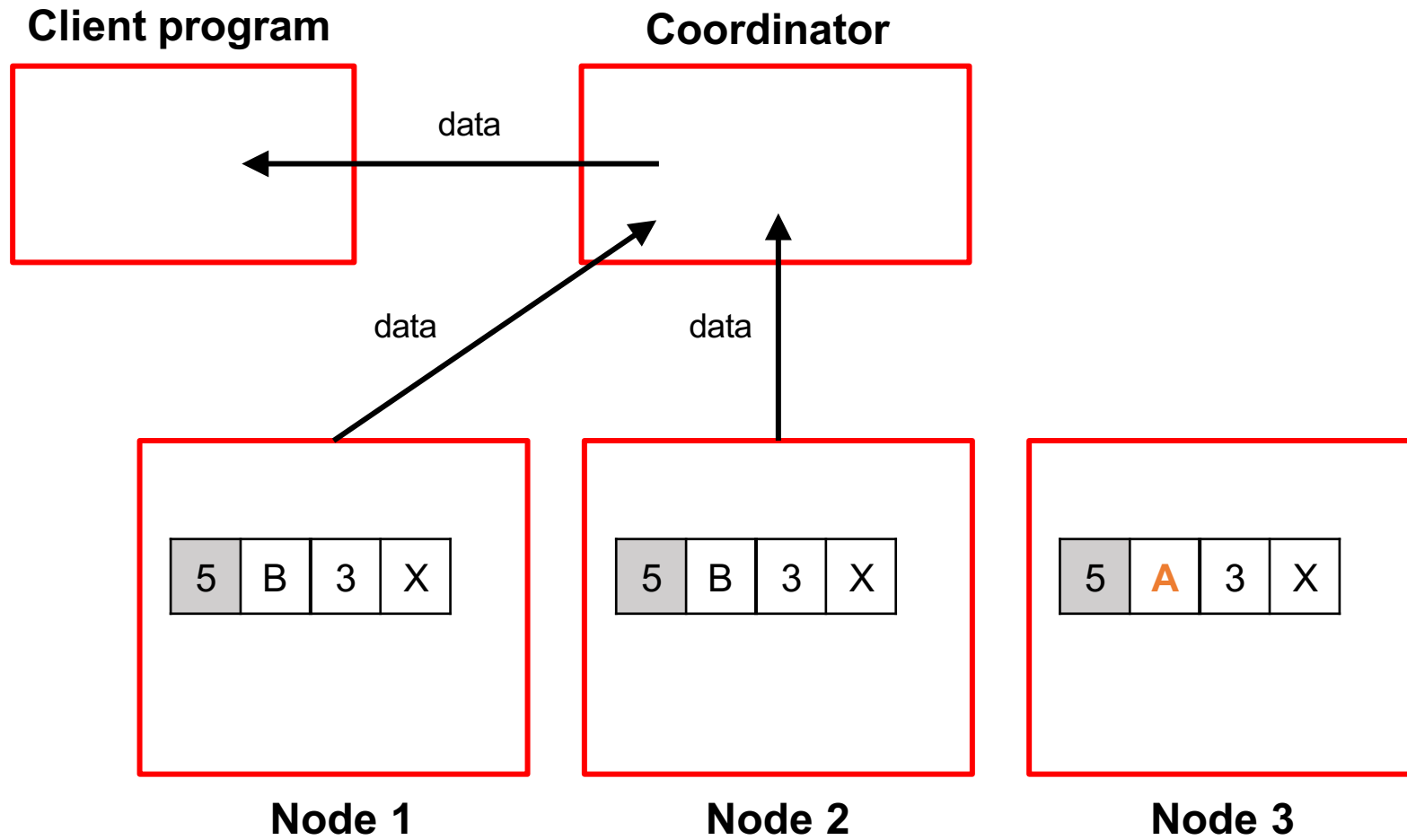
RF = 3



Read from **R** replicas (R is configurable). Here R = 2.
Hopefully at least one of the replicas has new data.

Dynamo reads

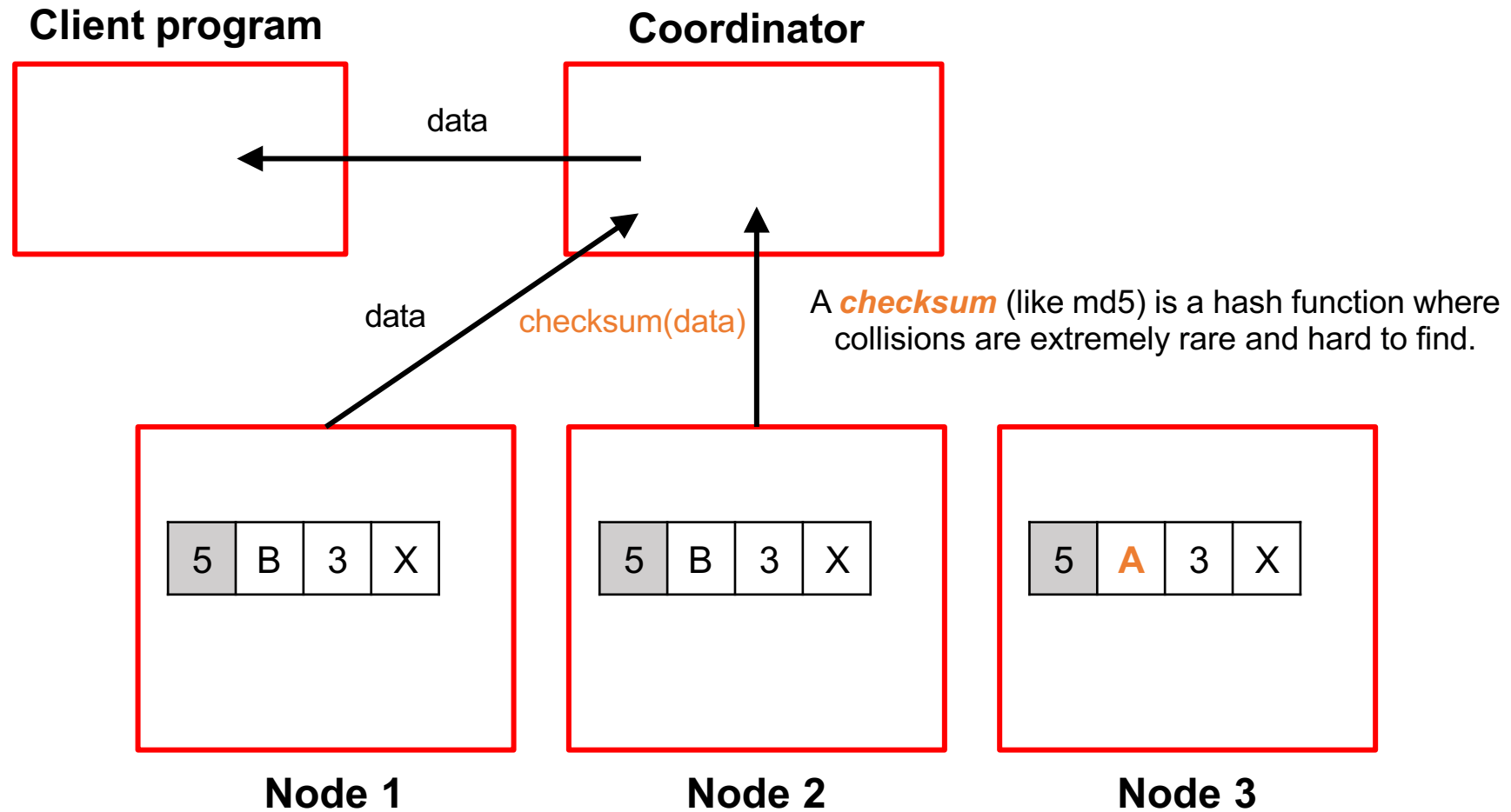
RF = 3



R = 2 means we'll often read identical data from two replicas (wasteful)

Dynamo reads

RF = 3

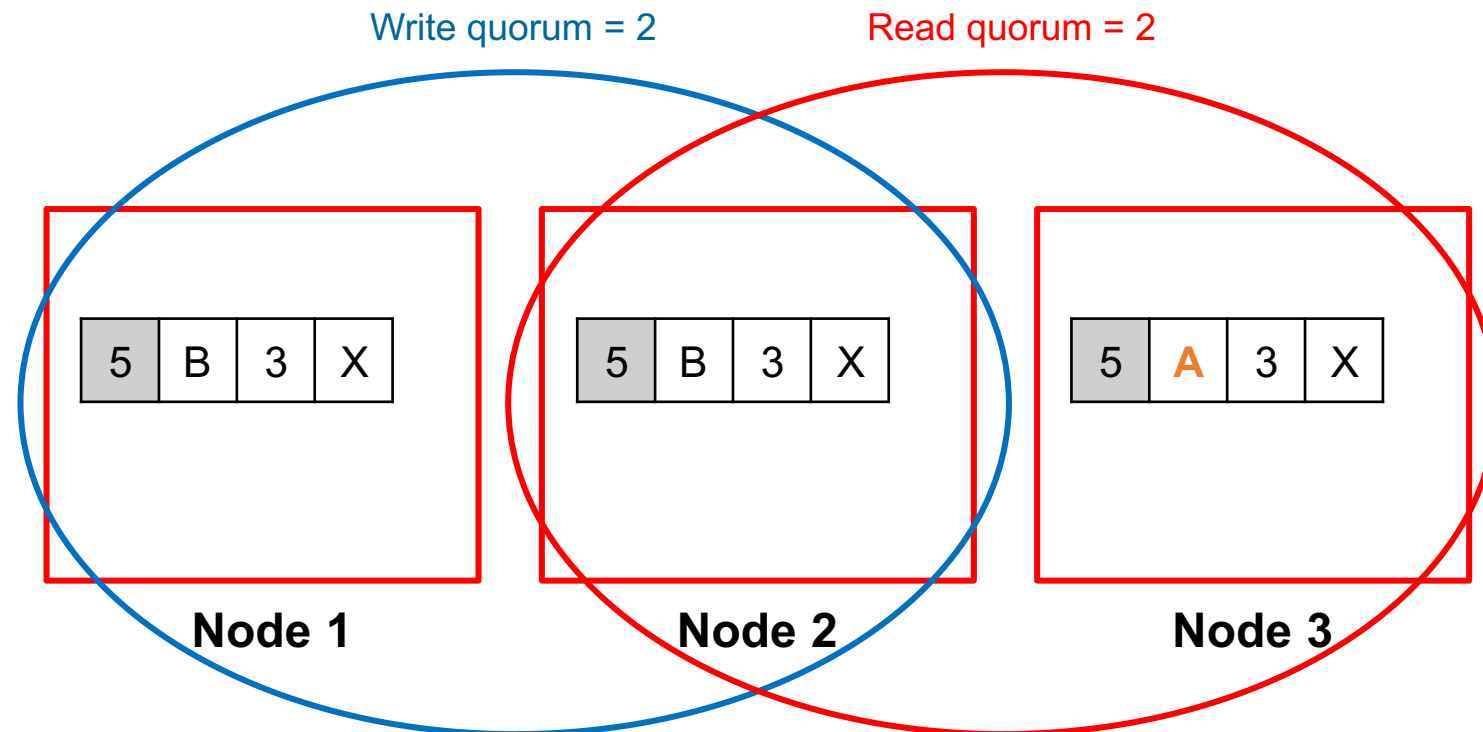


R = 2 means we'll often read identical data from two replicas (wasteful)
Optimization: Read one copy, and only request checksum from others.

When $R + W > RF$

$RF = 3$

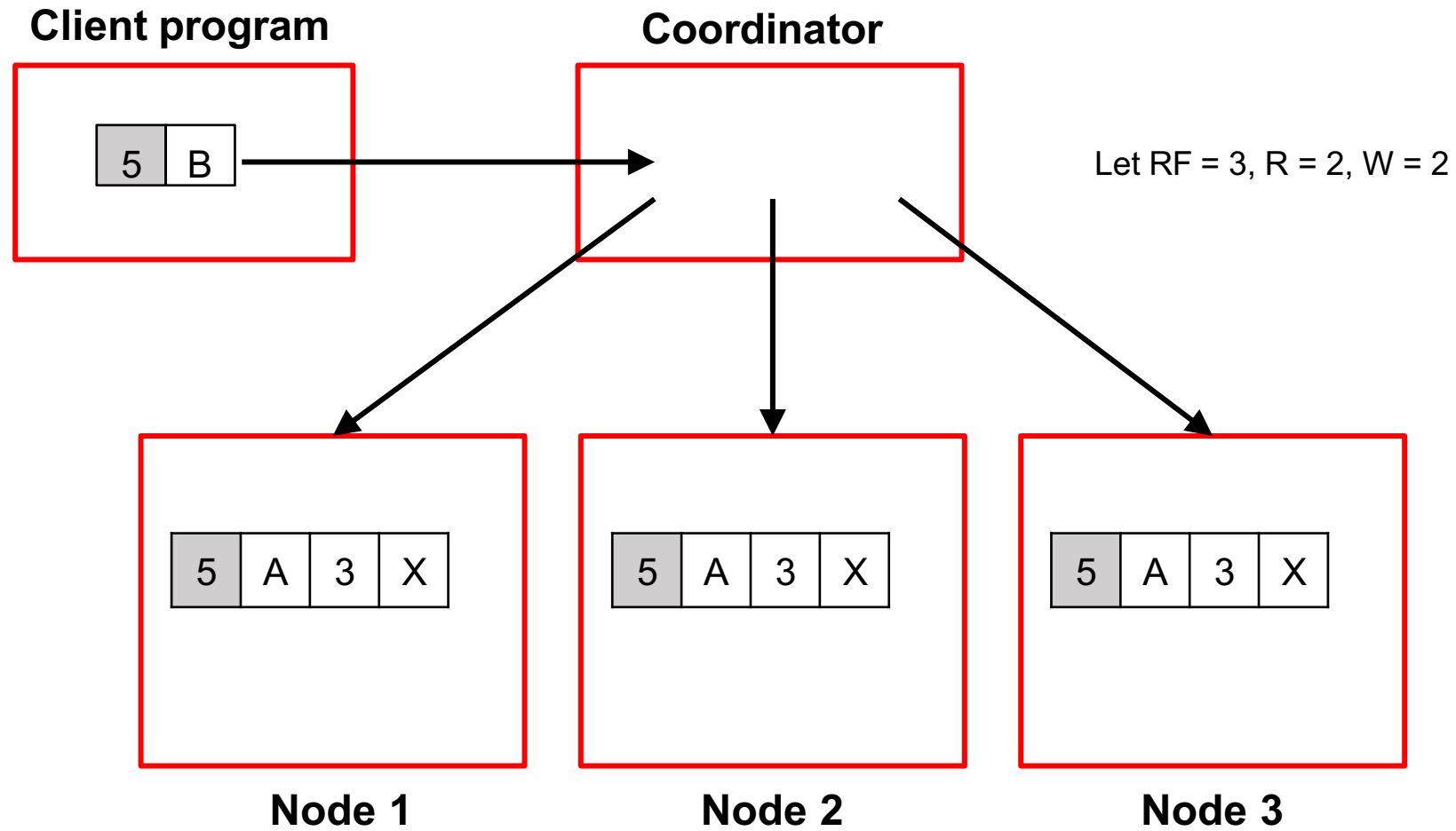
When $R + W > RF$, the replicas read + written will **overlap**.



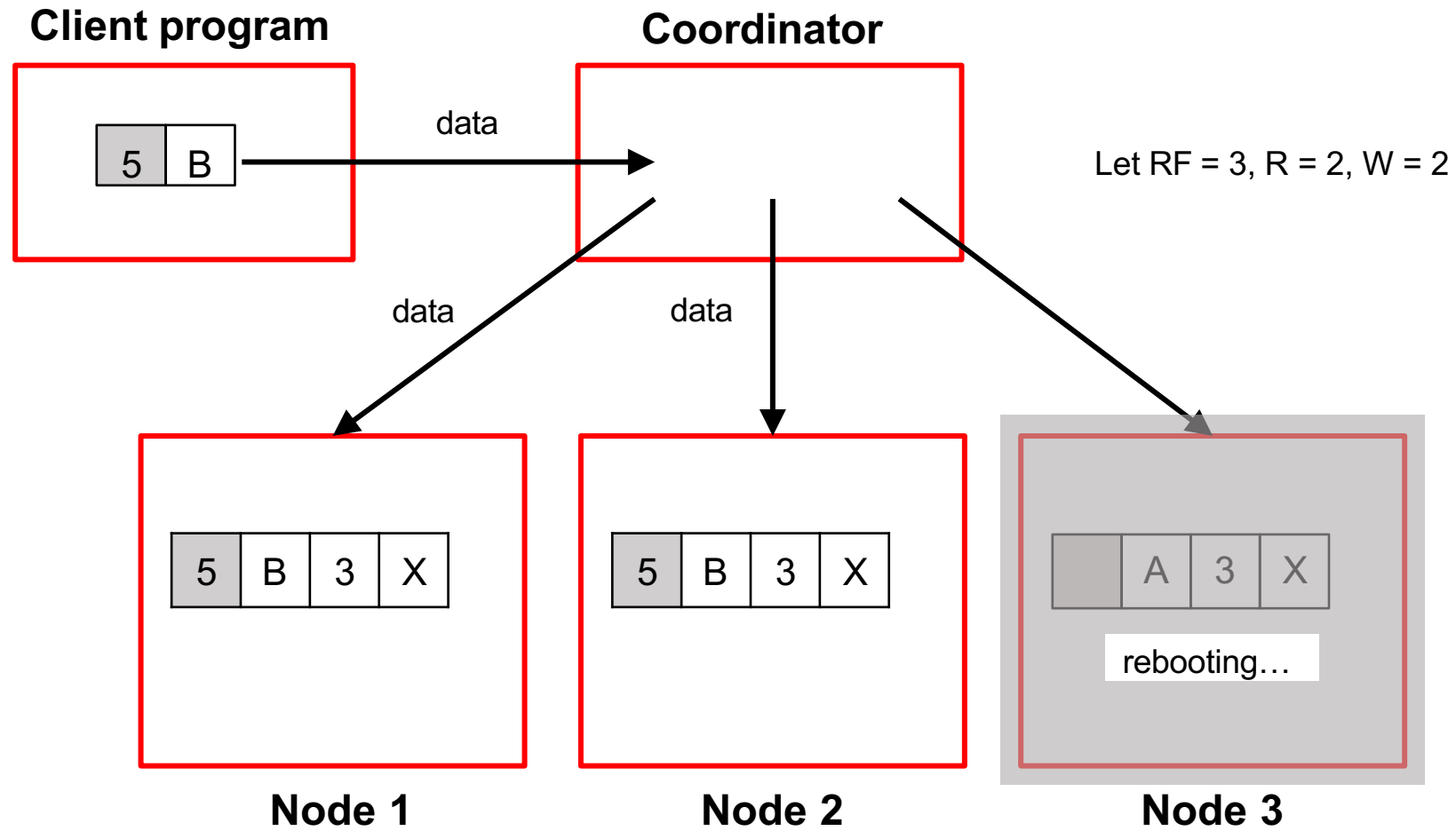
Tradeoff: Tuning R and W

RF	R	W	Behavior
3	2	2	Parameters from the Dynamo paper: Relatively balanced configuration; Good durability, good R/W latency
3	3	1	Slow reads, weak durability, fast writes Writes are highly available, therefore fast; Reads will not return data even if one node is down; reads may fail; Risk: If the one node that took the write fails permanently, we'll lose committed data.
3	1	3	Slow writes, strong durability, fast reads Reads are highly available, therefore fast; Writes are slow (from client's perspective) as they involve writing to three replicas.
3	3	3	More likely that reads see all prior writes
3	1	1	Read quorum doesn't overlap write quorum Speed + availability more important than consistency

Getting conflicting versions



Getting conflicting versions



Getting conflicting versions

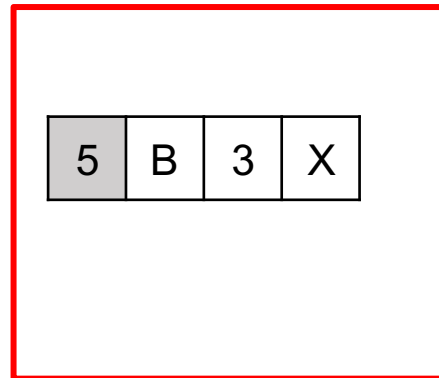
Client program



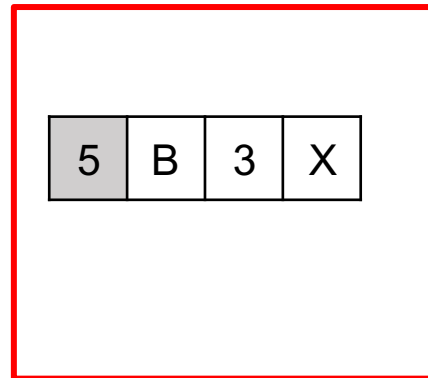
Coordinator



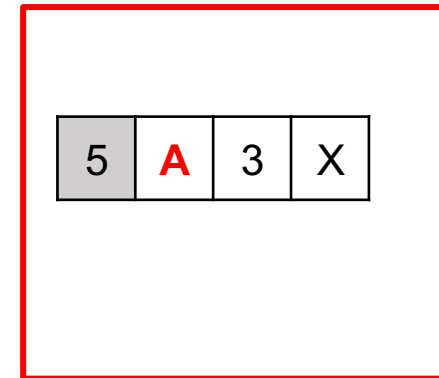
Let $RF = 3$, $R = 2$, $W = 2$



Node 1

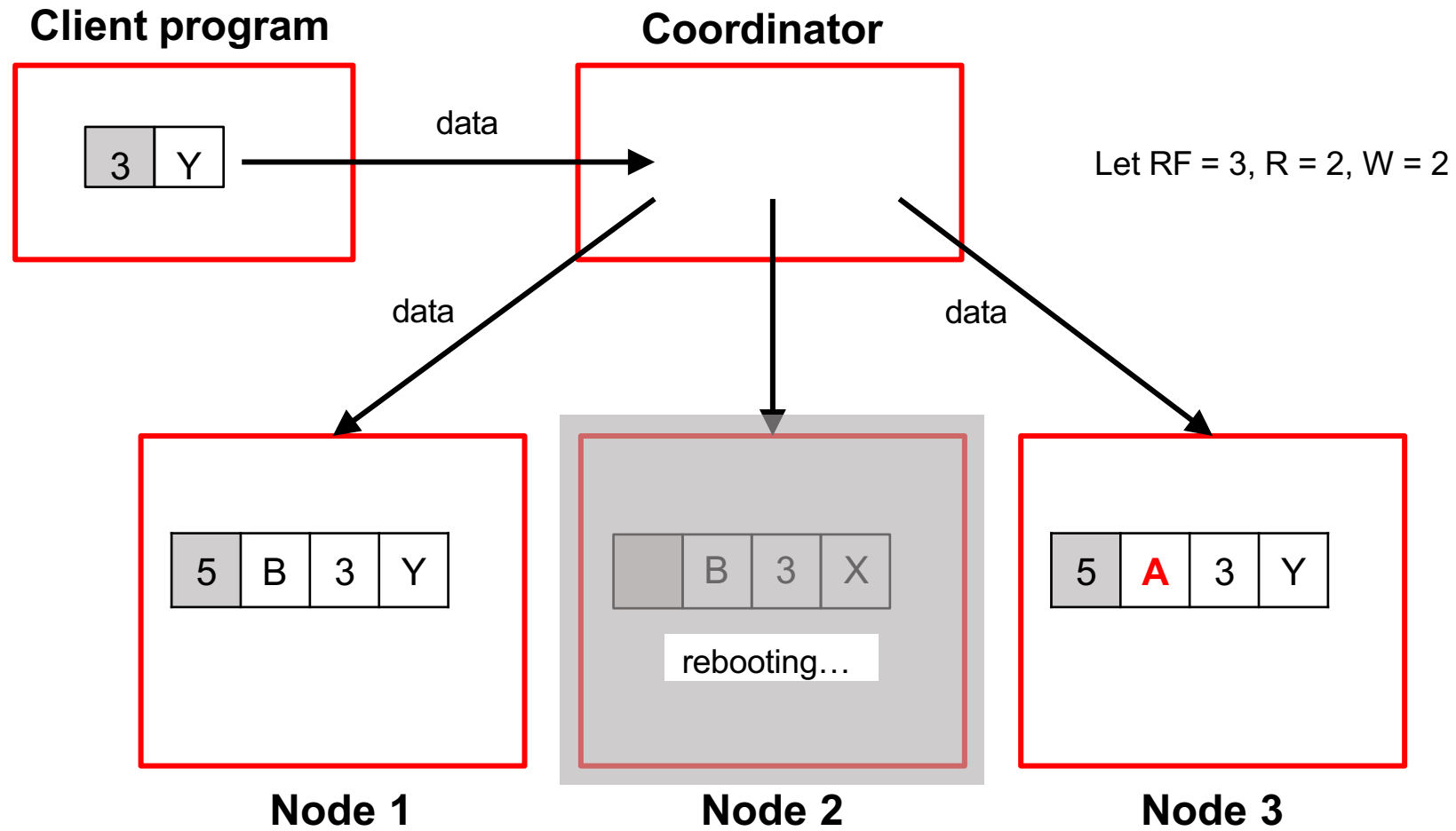


Node 2



Node 3

Getting conflicting versions



Getting conflicting versions

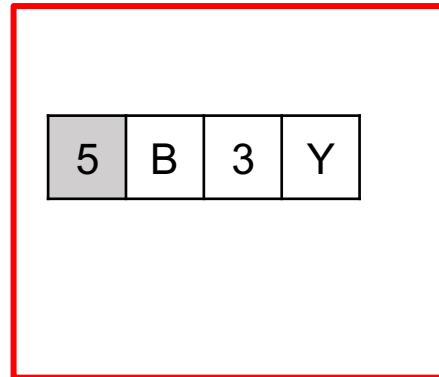
Client program



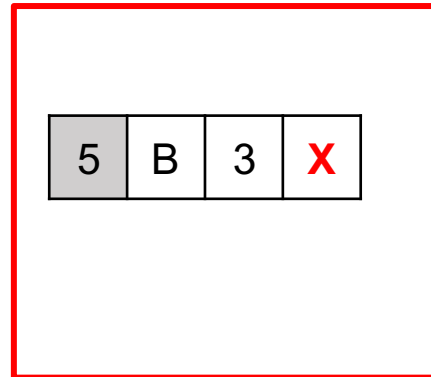
Coordinator



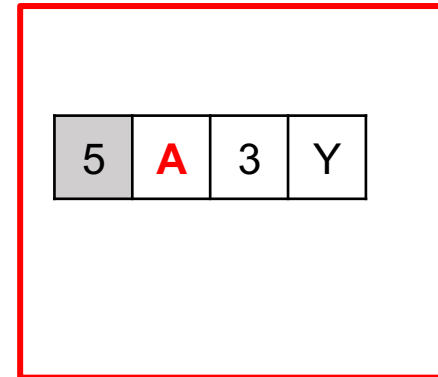
Let $RF = 3$, $R = 2$, $W = 2$



Node 1

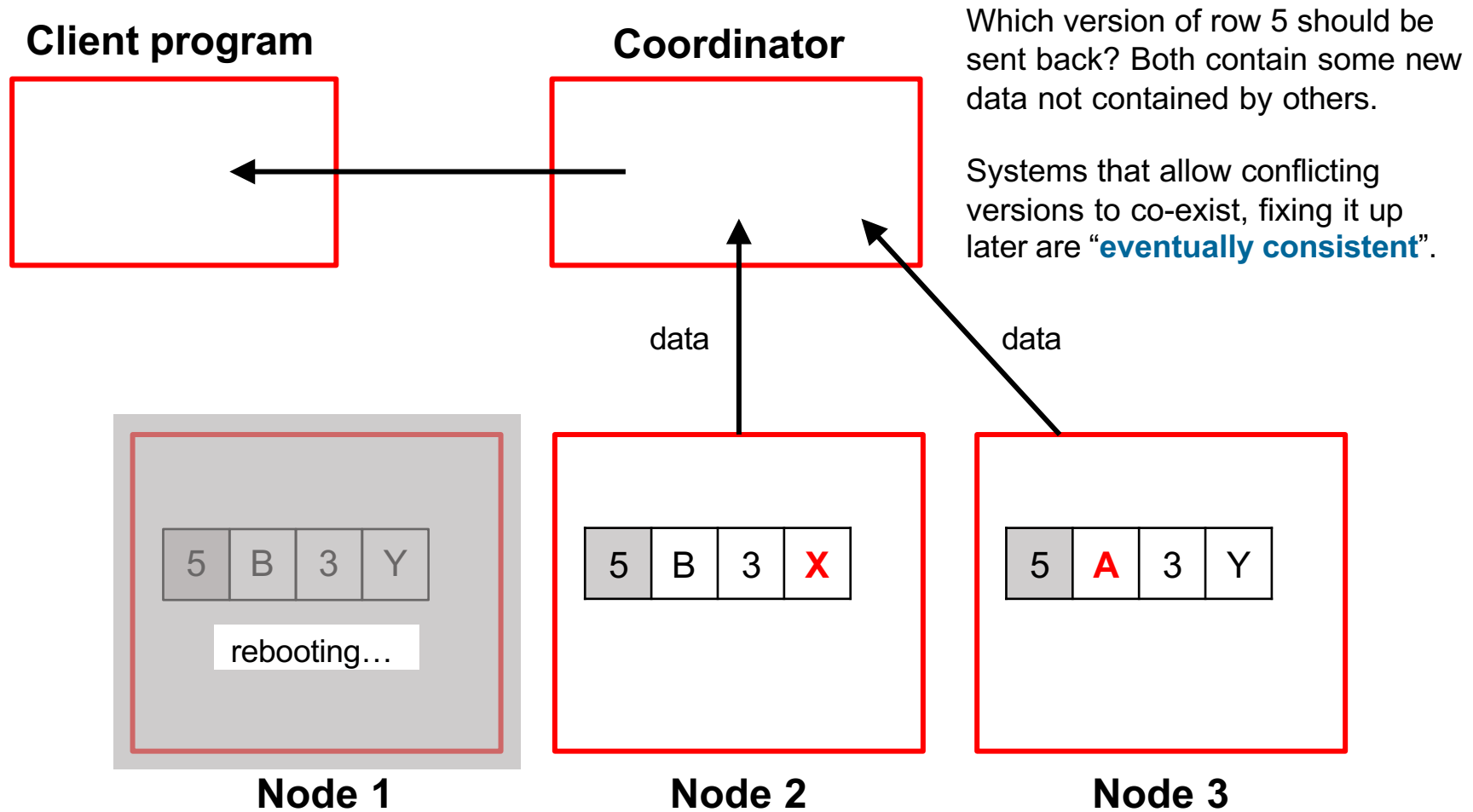


Node 2

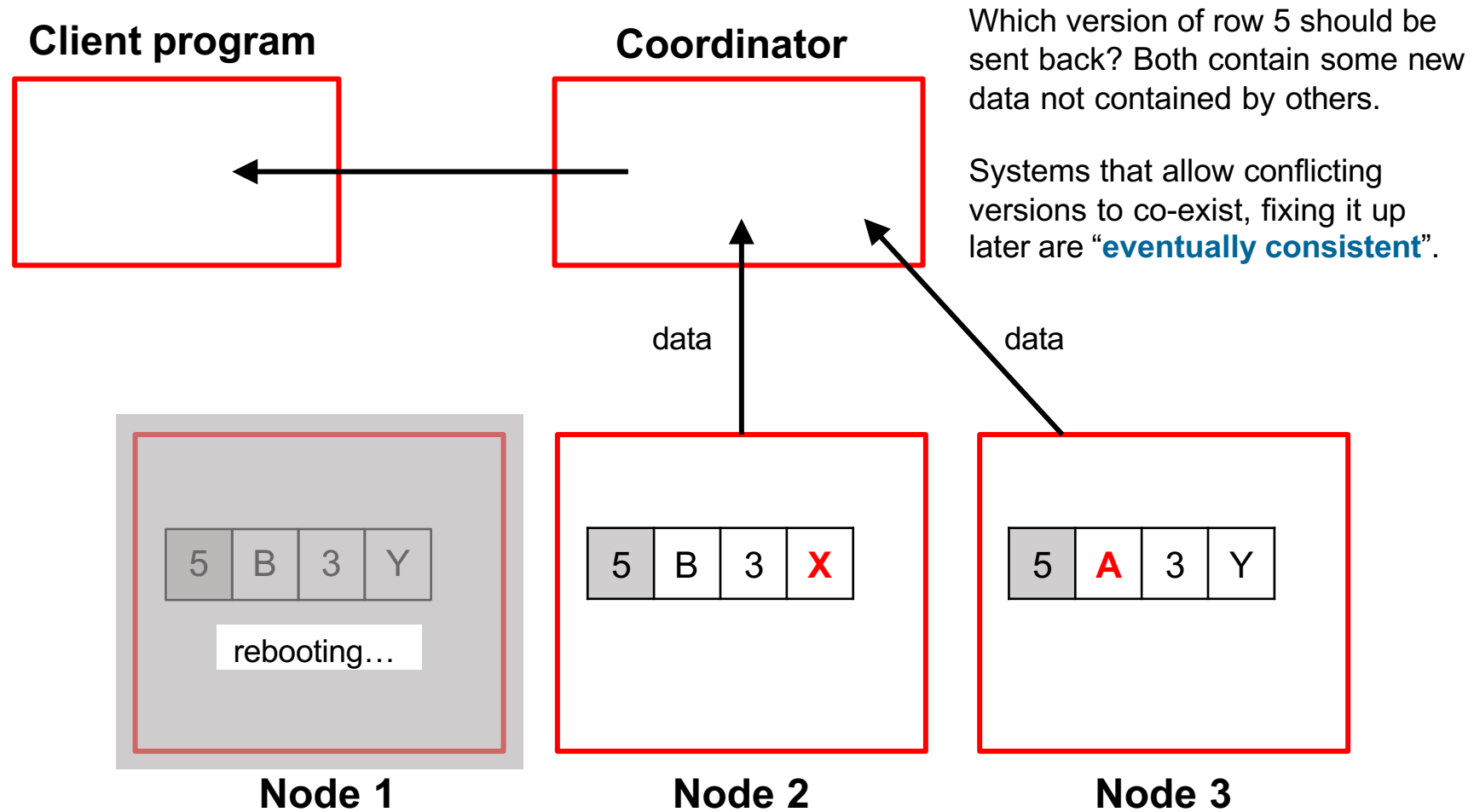


Node 3

Getting conflicting versions



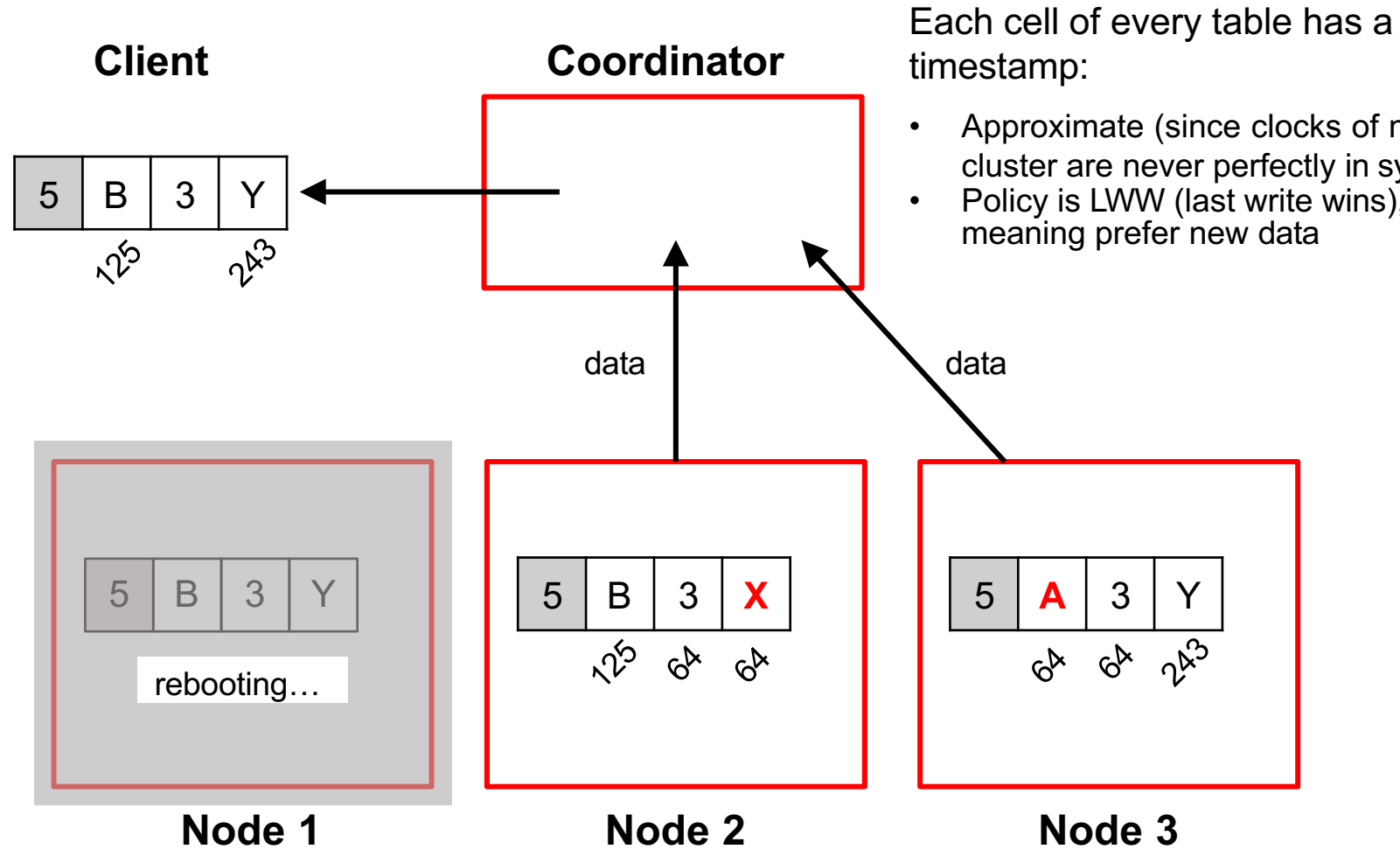
Getting conflicting versions



Approach:

- Send all versions back to client, which will need specialized conflict resolution code
- Automatically combine them into a new row, and write that (if possible to all replicas)

Timestamps (logical clock)



Dynamo API

- Basic interface is a key-value store
 - `get(k)` and `put(k, v)`
 - Keys and values opaque to Dynamo
- `get(key) → value, context`
 - Returns one value or multiple conflicting values
 - Context describes **version(s)** of value(s)
 Contains the (logical) timestamp info.
- `put(key, context, value) → "OK"`
 - Context indicates which **versions** this version supersedes or merges

Credits

- Some slides are adapted from course slides of DS 5110 in UVA